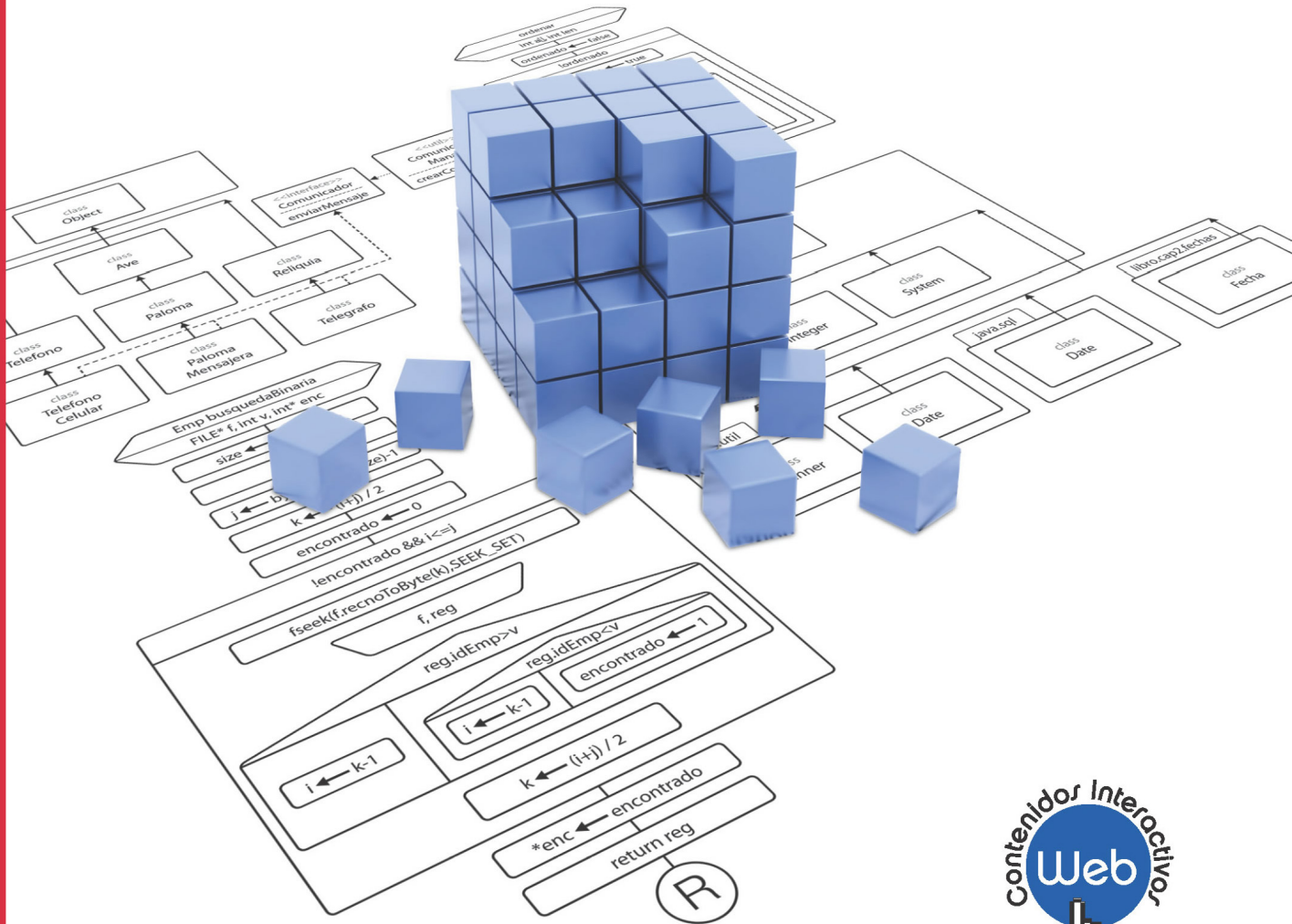


Algoritmos a fondo

CON IMPLEMENTACIONES EN C y JAVA
INCLUYE DIAGRAMAS DE FLUJO y UML

ING. PABLO AUGUSTO SZNAJDLEDER



Algoritmos a fondo

Con implementaciones en C y Java

Ing. Pablo Augusto Sznajdleder



Sznajdleder, Pablo
Algoritmos a fondo : con implementaciones en C y Java . - 1a ed. -
Buenos Aires : Alfaomega Grupo Editor Argentino, 2012
576 p. ; 24x21 cm.

ISBN 978-987-1609-37-6

1. Informática. I. Título
CDD 005.3

Queda prohibida la reproducción total o parcial de esta obra, su tratamiento informático y/o la transmisión por cualquier otra forma o medio sin autorización escrita de Alfaomega Grupo Editor Argentino S.A.

Edición: Damián Fernández

Revisión de estilo: Vanesa García y Juan Micán

Diagramación: Diego Ay

Revisión de armado: Vanesa García

Internet: <http://www.alfaomega.com.mx>

Todos los derechos reservados © 2012, por Alfaomega Grupo Editor Argentino S.A.

Paraguay 1307, PB, oficina 11

ISBN 978-987-1609-37-6

Queda hecho el depósito que prevé la ley 11.723

NOTA IMPORTANTE: La información contenida en esta obra tiene un fin exclusivamente didáctico y, por lo tanto, no está previsto su aprovechamiento a nivel profesional o industrial. Las indicaciones técnicas y programas incluidos han sido elaborados con gran cuidado por el autor y reproducidos bajo estrictas normas de control. Alfaomega Grupo Editor Argentino S.A. no será jurídicamente responsable por: errores u omisiones; daños y perjuicios que se pudieran atribuir al uso de la información comprendida en este libro, ni por la utilización indebida que pudiera dársele.

Los nombres comerciales que aparecen en este libro son marcas registradas de sus propietarios y se mencionan únicamente con fines didácticos, por lo que Alfaomega Grupo Editor Argentino S.A. no asume ninguna responsabilidad por el uso que se dé a esta información, ya que no infringe ningún derecho de registro de marca. Los datos de los ejemplos y pantallas son ficticios, a no ser que se especifique lo contrario.

Los hipervínculos a los que se hace referencia no necesariamente son administrados por la editorial, por lo que no somos responsables de sus contenidos o de su disponibilidad en línea.

Empresas del grupo:

Argentina: Alfaomega Grupo Editor Argentino, S.A.

Paraguay 1307 P.B. "11", Buenos Aires, Argentina, C.P. 1057

Tel.: (54-11) 4811-7183 / 0887 - E-mail: ventas@alfaomegaeditor.com.ar

México: Alfaomega Grupo Editor, S.A. de C.V.

Pitágoras 1139, Col. Del Valle, México, D.F., México, C.P. 03100

Tel.: (52-55) 5575-5022 - Fax: (52-55) 5575-2420 / 2490. Sin costo: 01-800-020-4396

E-mail: atencionalcliente@alfaomega.com.mx

Colombia: Alfaomega Colombiana S.A.

Carrera 15 No. 64 A 29, Bogotá, Colombia

PBX (57-1) 2100122 - Fax: (57-1) 6068648 - E-mail: cliente@alfaomega.com.co

Chile: Alfaomega Grupo Editor, S.A.

Doctor La Sierra 1437 - Providencia, Santiago, Chile

Tel.: (56-2) 235-4248 - Fax: (56-2) 235-5786 - E-mail: agechile@alfaomega.cl



Superman, por Octaviano Sznajdleder

A los amores de mi vida: mi esposa Analía y mi hijo Octaviano.
Ellos son el motor que impulsa todo lo que hago.

A la memoria de Naum, quien pasó a vivir en nuestros corazones.
Siempre me preguntaba: —¿Cómo va ese libro, Pablo?

Agradecimientos

A mi mamá Nélica, que no solo pone su casa a mi disposición sino que, además, siempre me prepara el té.

A mi editor y amigo Damián Fernández, que no para de ofrecerme inmejorables oportunidades.

A Graciela Sosisky y Domingo Mandrafina quienes, hace ya varios años, me dieron la oportunidad de incorporarme a la cátedra de Algoritmos.

A Adriana Adamoli por su colaboración y por el aporte de muchos de los ejercicios que se encuentran en la Web.

A Juan Grande quien, GTalk mediante, siempre estuvo presente para darme una mano.

A Marcelo Grillo, Gustavo Baez y a todos los promotores de Alfaomega por la amabilidad, la cordialidad y la excelente estadía que me han hecho pasar durante mi viaje a México.

A los maestros Alberto Templos Carbajal, Sergio Fuenlabrada y Edna Miranda por sus valiosísimos aportes.

Mensaje del Editor

Los conocimientos son esenciales en el desempeño profesional. Sin ellos es imposible lograr las habilidades para competir laboralmente. La universidad o las instituciones de formación para el trabajo ofrecen la oportunidad de adquirir conocimientos que serán aprovechados más adelante en beneficio propio y de la sociedad. El avance de la ciencia y de la técnica hace necesario actualizar continuamente esos conocimientos. Cuando se toma la decisión de embarcarse en una vida profesional, se adquiere un compromiso de por vida: mantenerse al día en los conocimientos del área u oficio que se ha decidido desempeñar.

Alfaomega tiene por misión ofrecerles a estudiantes y profesionales conocimientos actualizados dentro de lineamientos pedagógicos que faciliten su utilización y permitan desarrollar las competencias requeridas por una profesión determinada. Alfaomega espera ser su compañera profesional en este viaje de por vida por el mundo del conocimiento.

Alfaomega hace uso de los medios impresos tradicionales en combinación con las tecnologías de la información y las comunicaciones (IT) para facilitar el aprendizaje. Libros como este tienen su complemento en una página Web, en donde el alumno y su profesor encontrarán materiales adicionales, información actualizada, pruebas (test) de autoevaluación, diapositivas y vínculos con otros sitios Web relacionados.

Esta obra contiene numerosos gráficos, cuadros y otros recursos para despertar el interés del estudiante, y facilitarle la comprensión y apropiación del conocimiento.

Cada capítulo se desarrolla con argumentos presentados en forma sencilla y estructurada claramente hacia los objetivos y metas propuestas. Cada capítulo concluye con diversas actividades pedagógicas para asegurar la asimilación del conocimiento y su extensión y actualización futuras.

Los libros de Alfaomega están diseñados para ser utilizados dentro de los procesos de enseñanza-aprendizaje, y pueden ser usados como textos guía en diversos cursos o como apoyo para reforzar el desarrollo profesional.

Alfaomega espera contribuir así a la formación y el desarrollo de profesionales exitosos para beneficio de la sociedad.

Pablo Augusto Sznajdleder

Es Ingeniero en Sistemas de Información, egresado de la Universidad Tecnológica Nacional (UTN-FRBA) en 1999.

Actualmente, es profesor en la cátedra de “Algoritmos y Estructura de Datos” en la UTN-FRBA, pasando también por la Universidad Nacional de San Martín (UNSAM) y el Instituto de Tecnología ORT Argentina.

Trabajó como instructor Java para Sun Microsystems, Oracle e Informix/IBM entre otras empresas líderes.

Desde 1995 trabaja en sistemas, principalmente, en el desarrollo de aplicaciones empresariales distribuidas: primero en C/C++ y luego, en Java/JEE.

En 1996 comenzó a trabajar con Instructor Java para Sun Microsystems y, desde el 2000, se desempeña como consultor en la búsqueda y selección de RRHH capacitados en dicha tecnología, poniendo especial atención en la identificación de jóvenes estudiantes sin experiencia laboral previa, pero con gran potencial profesional.

Tiene las certificaciones internacionales *Sun Certified Java Programmer* (SCJP, 1997) y *Sun Certified Java Developer* (SCJD, 1998) y, además, está certificado como Instructor Oficial Java por Sun Microsystems (1997).

En el 2008 publicó *HolaMundo.pascal, Algoritmos y estructura de datos* cuyo contenido cubre por completo los temas que abarca la asignatura de igual nombre en UTN-FRBA. En el 2009 participó como revisor técnico en el libro *Análisis y diseño de algoritmos* (López, Jeder, Vega). En el 2010 publicó su libro sobre desarrollo de aplicaciones Java: *Java a fondo, Estudio del lenguaje y desarrollo de aplicaciones*.

**Revisor técnico: Alberto Templos Carbajal**

Es Ingeniero en computación de la Facultad de Ingeniería de la UNAM y ejerce, desde 1983, como Profesor de dicha facultad en las Divisiones de Ingeniería Eléctrica en distintas asignaturas de la carrera de Ingeniería en Computación y Educación Continua. Desde 1986, es Profesor a tiempo completo y ha ocupado los siguientes cargos: Coordinador de las carreras de Ingeniero en Computación y, de manera temporal, de Ingeniero Eléctrico Electrónico e Ingeniero en Telecomunicaciones, Jefe del Departamento de Ingeniería en Computación, Secretario Académico de las Divisiones de Ingeniería Eléctrica y Estudios de Postgrado y Coordinador de Postgrado en la Secretaría de Postgrado e Investigación de la Facultad de Ingeniería. También ha sido cofundador de dos empresas de computación y asesor en esa misma área de diversas compañías.

Actualmente, es miembro de diferentes comités, evaluador de planes de estudio del área de computación para diferentes organismos nacionales y es responsable de un proyecto de investigación e innovación tecnológica sobre el Diseño de algoritmos para robots. Su productividad, principalmente, está orientada a la docencia.

Contenido

Modulo 1

Programación estructurada

1. Introducción a los algoritmos y a la programación de computadoras	1	1.7.7 Las constantes	21
1.1 Introducción	2	1.7.7.1 La directiva de preprocesador #define	21
1.2 Concepto de algoritmo	2	1.7.7.2 El modificador const	22
1.2.1 Definición de algoritmo y problema	2	1.7.8 Nomenclatura para las constantes	22
1.2.2 Análisis del enunciado de un problema	3	1.8 Operadores aritméticos	22
1.2.2.1 Análisis del problema	3	1.8.1 Conversión de tipos de datos (type casting)	25
1.2.2.2 Datos de entrada	3	1.8.2 El operador % ("módulo" o "resto")	25
1.2.2.3 Datos de salida	4	1.8.3 Operadores relacionales	28
1.2.3 Memoria y operaciones aritméticas y lógicas	4	1.9 Expresiones lógicas	29
1.2.4 Teorema de la programación estructurada	4	1.9.1 Operadores lógicos	29
1.3 Conceptos de programación	5	1.10 Operadores de bits	30
1.3.1 Lenguajes de programación	5	1.10.1 Representación binaria de los tipos enteros	30
1.3.2 Codificación de un algoritmo	6	1.10.2 Operadores de desplazamiento de bits (>> y <<)	31
1.3.3 Bibliotecas de funciones	6	1.10.3 Representación hexadecimal	31
1.3.4 Programas de computación	6	Representación hexadecimal de números enteros negativos	32
1.3.5 Consola	7	1.10.4 Representación octal	32
1.3.6 Entrada y salida de datos	7	1.10.5 Operadores lógicos de bits	32
1.3.7 Lenguajes algorítmicos	7	1.11 Resumen	33
1.3.8 Pseudocódigo	7	1.12 Contenido de la página Web de apoyo	33
1.4 Representación gráfica de algoritmos	7	1.12.1 Mapa conceptual	33
1.4.1 Representación gráfica de la estructura secuencial o acción simple	8	1.12.2 Autoevaluaciones	33
1.4.2 Representación gráfica de la estructura de decisión	8	1.12.3 Videotutorial	33
1.4.3 Representación gráfica de la estructura de repetición	8	1.12.3.1 Instalación y uso de Eclipse para C	33
1.4.4 Representación gráfica de módulos o funciones	9	1.12.4 Presentaciones*	33
1.5 Nuestro primer programa	11	2. Estructuras básicas de control y lógica algorítmica	35
1.5.1 Codificación del algoritmo utilizando el lenguaje C	11	2.1 Introducción	36
1.5.2 El archivo de código fuente	12	2.2 Estructura secuencial	36
1.5.3 Comentarios en el código fuente	12	2.3 Estructura de decisión	36
1.5.4 La compilación y el programa ejecutable	13	2.3.1 Estructuras de decisión anidadas	38
1.5.5 El entorno integrado de desarrollo (IDE)	13	2.3.2 Selección en línea o if-inline	41
1.6 La memoria de la computadora	14	2.3.3 Macros	42
1.6.1 El byte	15	2.3.4 Selección múltiple (switch)	43
1.6.2 Conversión numérica: de base 2 a base 10	15	2.3.5 Asignación de valores alfanuméricos (función strcpy)	45
1.6.3 Dimensionamiento de los datos	15	2.4 Estructura de repetición	47
1.6.4 Los números negativos	16	2.4.1 Estructuras de repetición inexactas	47
1.6.5 Los caracteres	16	2.4.2 Estructuras de repetición exactas	49
1.7 Las variables	17	2.4.3 Contadores	51
1.7.1 Convención de nomenclatura para variables	17	2.4.4 Acumuladores	52
1.7.2 Los tipos de datos	18	2.4.5 Seguimiento del algoritmo y "prueba de escritorio"	53
1.7.3 Los tipos de datos provistos por el lenguaje C	18	2.4.6 El debugger, la herramienta de depuración	54
1.7.3.1 Notación húngara	19	2.4.7 Estructuras de repetición anidadas	57
1.7.4 La función de biblioteca printf	19	2.4.8 Manejo de valores booleanos	59
1.7.5 La función de biblioteca scanf	20	2.4.9 Máximos y mínimos	60
1.7.6 El operador de dirección &	21	2.5 Contextualización del problema	63
		2.6 Resumen	69

2.7	Contenido de la página Web de apoyo	69	4.2.2.2	Determinar si un carácter es una letra (función esLetra)	101
2.7.1	Mapa conceptual	69	4.2.2.3	Determinar si un carácter es una letra mayúscula o minúscula (funciones esMayuscula y esMinuscula)	102
2.7.2	Autoevaluaciones	69	4.2.2.4	Convertir un carácter a minúscula (función aMinuscula)	102
2.7.3	Videotutorial	69	4.2.2.5	Convertir un carácter a mayúscula (función aMayuscula)	102
2.7.3.1	Uso del debugger para depurar un programa	69			
2.7.4	Presentaciones*	69			
3.	Funciones, modularización y metodología top-down ...	71	4.3	Cadenas de caracteres	103
3.1	Introducción	72	4.3.1	El carácter '\0' (barra cero)	104
3.2	Conceptos iniciales	72	4.3.2	Longitud de una cadena	105
3.2.1	Metodología top-down	72	4.3.2.1	La cadena vacía	105
3.2.2	Módulos o subprogramas	72	4.4	Tratamiento de cadenas de caracteres	106
3.2.3	Funciones	72	4.4.1	Inicialización de una cadena de caracteres	106
3.2.4	Funciones de biblioteca	73	4.4.2	Funciones para el tratamiento de cadenas de caracteres	107
3.2.5	Invocación a funciones de biblioteca	73	4.4.2.1	Asignar o copiar una cadena a un char[] (función copiarCadena)	107
3.3	Funciones definidas por el programador	73	4.4.2.2	Determinar la longitud de una cadena (función longitud)	108
3.3.1	Prototipo de una función	74	4.4.2.3	Determinar si una cadena es "vacía" (función esVacia)	109
3.3.2	Invocar a una función	74	4.4.2.4	Concatenar cadenas (función concatenarCadena)	110
3.3.3	Desarrollo de una función	75	4.4.2.5	Comparar cadenas (función compararCadenas)	111
3.3.4	Convención de nomenclatura para funciones	76	4.4.2.6	Convertir cadenas a números enteros (función cadenaAEntero)	113
3.3.5	Funciones que no retornan ningún valor (tipo de datos void)	76	4.5	Funciones de biblioteca para manejo de cadenas	115
3.3.6	Archivos de cabecera (.h)	76	4.5.1	Otras funciones de biblioteca	116
3.3.7	Archivos de funciones (.c)	76	4.5.1.1	Dar formato a una cadena (función sprintf)	116
3.4	Legibilidad y reusabilidad del código	77	4.5.1.2	Interpretar (parsear) el formato de una cadena (función sscanf)	116
3.4.1	Abstracción	78	4.6	Resumen	117
3.4.2	Argumentos y parámetros	80	4.7	Contenido de la página Web de apoyo	117
3.5	Alcance de las variables (scope)	82	4.7.1	Mapa conceptual	117
3.5.1	Variables locales	82	4.7.2	Autoevaluaciones	117
3.5.2	Variables globales	83	4.7.3	Presentaciones*	117
3.6	Argumentos por valor y referencia	84	5.	Punteros a carácter	119
3.6.1	Punteros y direcciones de memoria	85	5.1	Introducción	120
3.6.2	El operador de indirección * (asterisco)	85	5.2	Conceptos iniciales	120
3.6.3	Argumentos por referencia	86	5.2.1	Aritmética de direcciones	121
3.6.4	Funciones que mantienen su estado	91	5.2.2	Prefijos y sufijos	122
3.6.5	Variables estáticas (modificador static)	94	5.2.2.1	Determinar si una cadena es prefijo de otra (función esPrefijo)	122
3.7	Resumen	96	5.2.2.2	Determinar si una cadena es sufijo de otra (función esSufijo)	123
3.8	Contenido de la página Web de apoyo	97	5.3	Funciones que retornan cadenas	124
3.8.1	Mapa conceptual	97	5.3.1	La función malloc	125
3.8.2	Autoevaluaciones	97	5.3.2	Subcadenas (función substring)	126
3.8.3	Videotutorial	97	5.3.2.1	Eliminar los espacios ubicados a la izquierda (función ltrim)	129
3.8.3.1	Mantener archivos de funciones separados del programa principal	97	5.3.2.2	Eliminar los espacios ubicados a la derecha (función rtrim)	130
3.8.4	Presentaciones*	97			
4.	Tipos de datos alfanuméricos	99			
4.1	Introducción	100			
4.2	Carácter	100			
4.2.1	El tipo de datos char	100			
4.2.2	Funciones para tratamiento de caracteres	101			
4.2.2.1	Determinar si un carácter es un dígito numérico (función esDigito)	101			

5.3.2.3	Eliminar los espacios en ambos extremos de la cadena (función trim)	131	7.4.2	Agregar un elemento al array	165
5.3.3	Función de biblioteca strtok	132	7.4.3	Búsqueda secuencial	166
5.4	Resumen	134	7.4.4	Buscar y agregar	168
5.5	Contenido de la página Web de apoyo	134	7.4.5	Insertar un elemento	172
5.5.1	Mapa conceptual	134	7.4.6	Eliminar un elemento	174
5.5.2	Autoevaluaciones	134	7.4.7	Insertar en orden	176
5.5.3	Presentaciones*	134	7.4.8	Buscar en orden	178
6.	Punteros, arrays y aritmética de direcciones	135	7.4.9	Buscar e insertar en orden	179
6.1	Introducción	136	7.4.10	Ordenar arrays (algoritmo de la "burbuja")	180
6.2	Punteros y direcciones de memoria	136	7.4.11	Búsqueda binaria o dicotómica	183
6.2.1	El operador de dirección &	136	7.4.11.1	Implementación	183
6.2.2	Los punteros	137	7.5	Arrays multidimensionales	188
6.2.3	El operador de indirección *	137	7.5.1	Arrays bidimensionales (matrices)	189
6.2.4	Funciones que reciben punteros	137	7.5.1.1	Recorrer una matriz por fila/columna	190
6.3	Arrays	139	7.5.1.2	Recorrer una matriz por columna/fila	190
6.3.1	La capacidad del array	139	7.5.2	Arrays tridimensionales (cubos)	192
6.3.2	Acceso a los elementos de un array	139	7.6	Tipos de datos definidos por el programador	192
6.3.3	Dimensionamiento e inicialización de arrays	141	7.6.1	Introducción al encapsulamiento a través de TADs	193
6.3.4	Crear arrays dinámicamente (funciones malloc y sizeof)	142	7.6.2	Estructuras o registros	195
6.3.5	Punteros genéricos void*	142	7.6.3	Representación gráfica de una estructura	196
6.4	Relación entre arrays y punteros	143	7.6.4	Estructuras anidadas	196
6.4.1	Aritmética de direcciones	143	7.6.5	Estructuras con campos de tipo array	197
6.5	Código compacto y eficiente	144	7.6.6	Punteros a estructuras	197
6.5.1	Operadores de incremento y decremento (operadores unarios)	144	7.6.6.1	El operador "flecha" ->	198
6.5.2	"Pre" y "post" incremento y decremento	144	7.6.7	Arrays de estructuras	198
6.5.3	Operadores de asignación	145	7.6.8	Estructuras con campos de tipo "array de estructuras"	199
6.5.4	Incremento de punteros	146	7.7	Resumen	199
6.5.4.1	Implementación compacta de la función copiarCadena	146	7.8	Contenido de la página Web de apoyo	200
6.5.4.2	Implementación compacta de la función longitud	147	7.8.1	Mapa conceptual	200
6.6	Arrays de cadenas	148	7.8.2	Autoevaluaciones	200
6.6.1	Argumentos en línea de comandos (int argc, char* argv[])	151	7.8.3	Videotutoriales	200
6.7	Resumen	153	7.8.3.1	Algoritmo de la burbuja	200
6.8	Contenido de la página Web de apoyo	153	7.8.3.2	Algoritmo de la búsqueda binaria	200
6.8.1	Mapa conceptual	153	7.8.4	Presentaciones*	200
6.8.2	Autoevaluaciones	153	8.	Operaciones sobre archivos	201
6.8.3	Videotutorial	153	8.1	Introducción	202
6.8.3.1	Pasar argumentos en línea de comandos con Eclipse	153	8.1.1	Memoria principal o memoria RAM de la computadora	202
6.8.4	Presentaciones*	153	8.1.2	Medios de almacenamiento (memoria secundaria)	202
7.	Tipos de datos estructurados	155	8.2	Archivos	202
7.1	Introducción	156	8.2.1	Abrir un archivo	203
7.2	Acceso directo sobre arrays	156	8.2.2	Escribir datos en un archivo	203
7.3	Acceso indirecto sobre arrays	163	8.2.3	Leer datos desde un archivo	204
7.4	Operaciones sobre arrays	164	8.2.4	El identificador de posición (puntero)	205
7.4.1	Capacidad vs. longitud de un array	164	8.2.5	Representación gráfica	206
			8.2.6	Valor actual del identificador de posición (función ftell)	206
			8.2.7	Manipular el valor del identificador de posición (función fseek)	207
			8.2.8	Calcular el tamaño de un archivo	208
			8.2.9	Archivos de texto vs. archivos binarios	209

8.3	Archivos de registros	209	10.3	Apareo de archivos	256
8.3.1	Archivos de estructuras	210	10.3.1	Apareo de archivos con corte de control	261
8.3.1.1	Grabar estructuras (registros) en un archivo	210	10.4	Resumen	266
8.3.1.2	Leer estructuras (registros) desde un archivo	211	10.5	Contenido de la página Web de apoyo	266
8.3.1.3	Legibilidad del código fuente.....	212	10.5.1	Mapa conceptual	266
8.3.2	Acceso directo a registros	213	10.5.2	Autoevaluaciones	266
8.3.2.1	Acceso directo para lectura.....	214	10.5.3	Presentaciones*	266
8.3.2.2	Acceso directo para escritura.....	215	11.	Estructuras de datos dinámicas lineales.....	267
8.3.2.3	Agregar un registro al final del archivo.....	216	11.1	Introducción	268
8.3.3	Calcular la cantidad de registros que tiene un archivo	217	11.2	Estructuras estáticas	269
8.4	Lectura y escritura en bloques (buffers).....	217	11.3	Estructuras dinámicas	269
8.5	Archivos de texto	219	11.3.1	El nodo	269
8.5.1	Apertura de un archivo de texto	220	11.4	Listas enlazadas	269
8.5.2	Leer y escribir caracteres (funciones getc y putc)	220	11.4.1	Estructuras de datos dinámicas lineales.....	270
8.5.3	Escribir líneas (función fprintf)	221	11.4.2	Estructuras de datos dinámicas no lineales	270
8.5.4	Leer líneas (función fgets).....	221	11.4.3	Punteros por referencia.....	270
8.5.5	Leer datos formateados (función fscanf)	222	11.5	Operaciones sobre listas enlazadas	271
8.6	Operaciones lógicas sobre archivos	223	11.5.1	Agregar un elemento nuevo al final de una lista	272
8.6.1	Limitaciones de los archivos secuenciales	223	11.5.2	Recorrer una lista para mostrar su contenido	276
8.6.2	Ordenamiento de archivos en memoria.....	224	11.5.3	Liberar la memoria que utilizan los nodos de una lista enlazada.....	276
8.6.3	Relación entre el número de byte y el número de registro	227	11.5.4	Determinar si la lista contiene un valor determinado	278
8.6.4	Búsqueda binaria sobre archivos	227	11.5.5	Eliminar un elemento de la lista.....	280
8.6.5	Indexación	228	11.5.6	Insertar un valor respetando el ordenamiento de la lista	283
8.6.6	Indexación de archivos	229	11.5.7	Insertar un valor solo si la lista aún no lo contiene	285
8.6.7	Eliminar registros en un archivo (bajas lógicas)	233	11.6	Estructura Pila (LIFO)	286
8.6.8	Bajas lógicas con soporte en un archivo auxiliar.....	235	11.6.1	Implementación de la estructura pila.....	286
8.7	Resumen	236	11.6.2	Operaciones poner (push) y sacar (pop).....	286
8.8	Contenido de la página Web de apoyo	236	11.6.3	Determinar si la pila tiene elementos o no	289
8.8.1	Mapa conceptual	236	11.7	Estructura Cola (FIFO).....	289
8.8.2	Autoevaluaciones	236	11.7.1	Lista enlazada circular	290
8.8.3	Videotutoriales.....	236	11.7.2	Implementar una cola sobre una lista circular	290
8.8.3.1	Leer y escribir un archivo.....	236	11.7.3	Operaciones encolar y desencolar	292
8.8.3.2	Leer y escribir un archivo de registros	236	11.8	Lista doblemente enlazada	294
8.8.4	Presentaciones*	236	11.9	Nodos que contienen múltiples datos.....	295
9.	Tipo Abstracto de Dato (TAD).....	237	11.9.1	Nodo con múltiples campos	295
9.1	Introducción		11.9.2	Nodo con un único valor de tipo struct	296
9.2	Capas de abstracción		11.10	Estructuras de datos combinadas	297
9.3	Tipos de datos		11.10.1	Lista y sublista.....	298
9.4	Resumen		11.10.2	Arrays de colas.....	300
9.5	Contenido de la página Web de apoyo		11.10.3	Matriz de pilas	307
10.	Análisis de ejercicios integradores	239	11.11	Resumen	311
10.1	Introducción	240	11.12	Contenido de la página Web de apoyo	311
10.2	Problemas con corte de control.....	240	11.12.1	Mapa conceptual	311
10.2.1	Archivos de novedades vs. archivos maestros.....	246	11.12.2	Autoevaluaciones	311
10.2.2	Uso de arrays auxiliares	249	11.12.3	Presentaciones*	311
10.2.3	Mantener archivos (pequeños) en memoria.....	250			



Modulo 2

Programación orientada a objetos

12. Encapsulamiento a través de clases y objetos	313
12.1 Introducción	314
12.2 Clases y objetos	314
12.2.1 Las clases	314
12.2.2 Miembros de la clase	315
12.2.3 Interfaz y encapsulamiento	316
12.2.4 Estructura de una clase	317
12.2.5 El constructor y el destructor	318
12.2.6 Los métodos	319
12.2.7 Los objetos	319
12.2.8 Instanciar objetos	320
12.2.9 Operadores new, delete y punteros a objetos	320
12.2.10 Sobrecarga de métodos	320
12.3 Encapsulamiento de estructuras lineales	321
12.3.1 Análisis de la clase Pila	321
12.3.2 Templates y generalizaciones	323
12.4 El lenguaje de programación Java	326
12.4.1 El programa principal en Java	328
12.4.2 Templates en C++, generics en Java	328
12.4.3 Los wrappers (envoltorios) de los tipos de datos primitivos	330
12.4.4 Autoboxing	330
12.5 Resumen	331
12.6 Contenido de la página Web de apoyo	331
12.6.1 Mapa conceptual	331
12.6.2 Autoevaluaciones	331
12.6.3 Presentaciones*	331
13. Introducción al lenguaje de programación Java	333
13.1 Introducción	334
13.2 Comencemos a programar	334
13.2.1 El Entorno Integrado de Desarrollo (IDE)	335
13.2.2 Entrada y salida estándar	335
13.2.3 Comentarios en el código fuente	336
13.3 Tipos de datos, operadores y estructuras de control	337
13.3.1 El bit de signo para los tipos de datos enteros	337
13.3.2 El compilador y la máquina virtual (JVM o JRE)	337
13.3.3 Estructuras de decisión	337
13.3.4 Estructuras iterativas	340
13.3.5 El tipo de datos boolean y las expresiones lógicas	342
13.3.6 Las constantes	343
13.3.7 Arrays	344
13.3.8 Matrices	346
13.3.9 Literales de cadenas de caracteres	348
13.3.10 Caracteres especiales	350
13.3.11 Argumentos en línea de comandos	351
13.4 Tratamiento de cadenas de caracteres	351
13.4.1 Acceso a los caracteres de un string	351
13.4.2 Mayúsculas y minúsculas	352

13.4.3 Ocurrencias de caracteres	353
13.4.4 Subcadenas	353
13.4.5 Prefijos y sufijos	354
13.4.6 Posición de un substring dentro de la cadena	354
13.4.7 Conversión entre números y cadenas	355
13.4.8 Representación en diferentes bases numéricas	355
13.4.9 La clase StringTokenizer	356
13.4.10 Comparación de cadenas	357
13.5 Resumen	359
13.6 Contenido de la página Web de apoyo	359
13.6.1 Mapa conceptual	359
13.6.2 Autoevaluaciones	359
13.6.3 Videotutorial	359
13.6.3.1 Instalar y utilizar Eclipse para Java	359
13.6.4 Presentaciones*	359
14. Programación orientada a objetos	361
14.1 Introducción	362
14.2 Clases y objetos	362
14.2.1 Los métodos	363
14.2.2 Herencia y sobrescritura de métodos	365
14.2.3 El método toString	365
14.2.4 El método equals	366
14.2.5 Declarar y "crear" objetos	367
14.2.6 El constructor	368
14.2.7 Repaso de lo visto hasta aquí	369
14.2.8 Convenciones de nomenclatura	370
14.2.8.1 Los nombres de las clases	370
14.2.8.2 Los nombres de los métodos	370
14.2.8.3 Los nombres de los atributos	371
14.2.8.4 Los nombres de las variables de instancia	371
14.2.8.5 Los nombres de las constantes	371
14.2.9 Sobrecarga de métodos	371
14.2.10 Encapsulamiento	374
14.2.11 Visibilidad de los métodos y los atributos	376
14.2.12 Packages (paquetes)	377
14.2.13 Estructura de paquetes y la variable CLASSPATH	377
14.2.14 Las APIs (Application Programming Interface)	378
14.2.15 Representación gráfica UML	379
14.2.16 Importar clases de otros paquetes	380
14.3 Herencia y polimorfismo	380
14.3.1 Polimorfismo	383
14.3.2 Constructores de subclases	385
14.3.3 La referencia super	385
14.3.4 La referencia this	388
14.3.5 Clases abstractas	389
14.3.6 Constructores de clases abstractas	392
14.3.7 Instancias	395
14.3.8 Variables de instancia	396
14.3.9 Variables de la clase	399
14.3.10 El garbage collector (recolector de residuos)	399
14.3.11 El método finalize	399

14.3.12	Constantes.....	400
14.3.13	Métodos de la clase.....	400
14.3.14	Clases utilitarias.....	403
14.3.15	Referencias estáticas.....	403
14.3.16	Colecciones (primera parte).....	404
14.3.17	Clases genéricas.....	409
14.4	Interfaces.....	412
14.4.1	Desacoplamiento de clases.....	413
14.4.2	El patrón de diseño de la factoría de objetos.....	415
14.4.3	Abstracción a través de interfaces.....	416
14.4.4	La interface Comparable.....	416
14.4.5	Desacoplar aún más.....	420
14.4.6	La interface Comparator.....	423
14.5	Colecciones de objetos.....	424
14.5.1	Cambio de implementación.....	426
14.5.2	El método Collections.sort.....	427
14.6	Excepciones.....	430
14.6.1	Errores lógicos vs. errores físicos.....	433
14.6.2	Excepciones declarativas y no declarativas.....	433
14.6.3	El bloque try-catch-finally.....	435
14.6.4	El método printStackTrace.....	438
14.7	Resumen.....	438
14.8	Contenido de la página Web de apoyo.....	438
14.8.1	Mapa conceptual.....	438
14.8.2	Autoevaluaciones.....	438
14.8.3	Videotutorial.....	438
14.8.3.1	Uso del javadoc.....	438
14.8.4	Presentaciones*.....	438
15.	Estructuras de datos dinámicas lineales en Java.....	439
15.1	Introducción.....	440
15.2	Listas (implementaciones de List).....	440
15.2.1	La clase ArrayList.....	440
15.2.2	La clase LinkedList.....	442
15.2.3	Comparación entre ArrayList y LinkedList.....	443
15.2.4	Desarrollo de la clase Performance.....	443
15.2.5	Introducción al análisis de complejidad algorítmica.....	444
15.2.5.1	Acceso aleatorio a los elementos de la colección.....	445
15.2.5.2	Eliminar un elemento de la colección.....	446
15.2.5.3	Insertar un elemento en la colección.....	448
15.2.6	Pilas y colas.....	448
15.3	Mapas (implementaciones de Map).....	449
15.3.1	Tablas de dispersión (Hashtable).....	449
15.3.2	Iterar una hashtable.....	451
15.3.3	Iterar una hashtable respetando el orden en que se agregaron los datos.....	452
15.4	Estructuras de datos combinadas.....	454
15.4.1	Ejemplo de una situación real.....	455
15.5	Resumen.....	459
15.6	Contenido de la página Web de apoyo.....	459
15.6.1	Mapa conceptual.....	459
15.6.2	Autoevaluaciones.....	459
15.6.3	Presentaciones*.....	459

Modulo 3

Aplicación práctica

16.	Compresión de archivos mediante el algoritmo de Huffman.....	461
16.1	Introducción.....	
16.2	El algoritmo de Huffman.....	
16.3	Aplicación práctica.....	
16.4	Análisis de clases y objetos.....	
16.5	Interfaces e implementaciones.....	
16.6	Manejo de archivos en Java.....	
16.7	Clases utilitarias.....	
16.8	Resumen.....	
16.9	Contenido de la página Web de apoyo.....	



Modulo 4

Conceptos avanzados

17.	Recursividad.....	463
17.1	Introducción.....	464
17.2	Conceptos iniciales.....	464
17.2.1	Funciones recursivas.....	464
17.2.2	Finalización de la recursión.....	465
17.2.3	Invocación a funciones recursivas.....	465
17.2.4	Funcionamiento de la pila de llamadas (stack).....	467
17.2.5	Funciones recursivas vs. funciones iterativas.....	470
17.3	Otros ejemplos de recursividad.....	471
17.4	Permutar los caracteres de una cadena.....	472
17.5	Búsqueda binaria.....	476
17.6	Ordenamiento por selección.....	478
17.7	La función de Fibonacci.....	480
17.7.1	Optimización del algoritmo recursivo de Fibonacci.....	485
17.8	Resumen.....	486
17.9	Contenido de la página Web de apoyo.....	486
17.9.1	Mapa conceptual.....	486
17.9.2	Autoevaluaciones.....	486
17.9.3	Presentaciones*.....	486
18.	Árboles.....	487
18.1	Introducción.....	488
18.1.1	Tipos de árbol.....	488
18.1.2	Implementación de la estructura de datos.....	488
18.2	Árbol binario.....	489
18.2.1	Niveles de un árbol binario.....	490
18.2.2	Recorrer los nodos de un árbol binario.....	490
18.2.3	Recorrido en amplitud o "por niveles".....	490
18.2.4	Recorridos en profundidad (preorden, postorden e inorden).....	490
18.2.5	Implementación iterativa del recorrido en preorden.....	492
18.2.6	Implementación iterativa del recorrido en postorden.....	493

Información del contenido de la página Web

El material marcado con asterisco (*) solo está disponible para docentes.

Capítulo 1

Introducción a los algoritmos y a la programación de computadoras

- Mapa conceptual
- Autoevaluación
- Videotutorial:
 - Instalación y uso de Eclipse para C
- Presentaciones*

Capítulo 2

Estructuras básicas de control y lógica algorítmica

- Mapa conceptual
- Autoevaluación
- Videotutorial:
 - Uso del debugger para depurar un programa
- Presentaciones*

Capítulo 3

Funciones, modularización y metodología top-down

- Mapa conceptual
- Autoevaluación
- Videotutorial:
 - Mantener archivos de funciones separados del programa principal
- Presentaciones*

Capítulo 4

Tipos de datos alfanuméricos

- Mapa conceptual
- Autoevaluación
- Presentaciones*

Capítulo 5

Punteros a carácter

- Mapa conceptual
- Autoevaluación
- Presentaciones*

Capítulo 6

Punteros, arrays y aritmética de direcciones

- Mapa conceptual
- Autoevaluación
- Videotutorial:
 - Pasar argumentos en línea de comandos con Eclipse

- Presentaciones*

Capítulo 7

Tipos de datos estructurados

- Mapa conceptual
- Autoevaluación
- Videotutoriales:
 - Algoritmo de la burbuja
 - Algoritmo de la búsqueda binaria
- Presentaciones*

Capítulo 8

Operaciones sobre archivos

- Mapa conceptual
- Autoevaluación
- Videotutoriales:
 - Leer y escribir un archivo
 - Leer y escribir un archivo de registros
- Presentaciones*

Capítulo 9

Tipo Abstracto de Dato (TAD)

- Mapa conceptual
- Autoevaluación
- Presentaciones*

Capítulo 10

Análisis de ejercicios integradores

- Mapa conceptual
- Autoevaluación
- Presentaciones*

Capítulo 11

Estructuras de datos dinámicas lineales

- Mapa conceptual
- Autoevaluación
- Presentaciones*

Capítulo 12

Encapsulamiento a través de clases y objetos

- Mapa conceptual
- Autoevaluación
- Presentaciones*

Capítulo 13**Introducción al lenguaje de programación Java**

- Mapa conceptual
- Autoevaluación
- Videotutorial:
 - Instalar y utilizar Eclipse para Java
- Presentaciones*

Capítulo 14**Programación orientada a objetos**

- Mapa conceptual
- Autoevaluación
- Videotutorial:
 - Uso del javadoc
- Presentaciones*

Capítulo 15**Estructuras de datos dinámicas lineales en Java**

- Mapa conceptual
- Autoevaluación
- Presentaciones*

Capítulo 16**Compresión de archivos mediante el algoritmo de Huffman**

- Mapa conceptual
- Autoevaluación
- Videotutorial:
 - Algoritmo de Huffman
- Presentaciones*

Capítulo 17**Recursividad**

- Mapa conceptual
- Autoevaluación
- Presentaciones*

Capítulo 18**Árboles**

- Mapa conceptual
- Autoevaluación
- Presentaciones*

Capítulo 19**Complejidad algorítmica**

- Mapa conceptual
- Autoevaluación
- Videotutorial:
 - Algoritmo heapsort, ordenamiento por montículos
- Presentaciones*

Capítulo 20**Algoritmos de ordenamiento**

- Mapa conceptual
- Autoevaluación
- Presentaciones*

Capítulo 21**Estrategia algorítmica**

- Mapa conceptual
- Autoevaluación
- Videotutorial:
 - Problema de los billetes por programación dinámica
- Presentaciones*

Capítulo 22**Algoritmos sobre grafos**

- Mapa conceptual
- Autoevaluación
- Videotutoriales:
 - Algoritmo de Dijkstra por greedy
 - Algoritmo de Dijkstra por dinámica
 - Algoritmo de Prim
 - Algoritmo de Kruskal
- Presentaciones*

Código fuente de cada capítulo**Hipervínculos de interés****Fe de erratas****Guía para el docente de las competencias específicas que se desarrollan con este libro ***

Registro en la Web de apoyo

Para tener acceso al material de la página Web de apoyo del libro:

1. Ir a la página <http://virtual.alfaomega.com.mx>
 2. Registrarse como usuario del sitio y propietario del libro.
 3. Ingresar al apartado de inscripción de libros y registrar la siguiente clave de acceso
-
4. Para navegar en la plataforma virtual de recursos del libro, usar los nombres de Usuario y Contraseña definidos en el punto número dos. El acceso a estos recursos es limitado. Si quiere un número extra de accesos, escriba a webmaster@alfaomega.com.mx

Estimado profesor: Si desea acceder a los contenidos exclusivos para docentes, por favor contacte al representante de la editorial que lo suele visitar o escribanos a:

webmaster@alfaomega.com.mx

	Videotutoriales sobre temas complementarios de la obra.
	Conceptos para recordar: bajo este icono se encuentran definiciones importantes que refuerzan lo explicado en la página.
	Comentarios o información extra: este ícono ayuda a comprender mejor o ampliar el texto principal
	Contenidos interactivos: indica la presencia de contenidos extra en la Web.

Prólogo

Cuando llegó a nuestras manos el libro de *Algoritmos a fondo* escrito por el Ing. Pablo Augusto Sznajdleder, esperábamos un documento con explicaciones complejas como la mayoría de las obras que abordan el tema pero, por el contrario, conforme leíamos el manuscrito nos encontramos con un texto ameno que lleva al lector en un proceso **paulatino, teórico-metodológico y persistente** en la construcción del conocimiento.

Una construcción del conocimiento **acompañada**, ya que para el autor un tema es verdadero si, y solo si, lleva un proceso de reflexión, que permite evidenciar que el conocimiento es indudable, provocando en el lector pocos sobresaltos y dudas.

El autor conduce con orden sus razonamientos, empezando por los conceptos más simples y fáciles de comprender para ascender poco a poco, de forma gradual, hasta llegar al conocimiento de lo más complejo, e incluso suponiendo un orden entre los que no se preceden naturalmente, provocando en los estudiantes el **rigor teórico-metodológico** que se espera en los especialistas de las áreas de informática y computación.

El texto es **persistente** ya que el autor cuida el hacer recuentos integrales del conocimiento y revisiones amplias de lo aprendido, que le permiten al lector estar seguro de que no se omitió algún aspecto del tema tratado, lo cual evita aburrir al lector con repeticiones innecesarias.

Además, resalta el uso de los diagramas Chapin, que son una versión modificada de los diagramas Nassi-Shneiderman, que como técnica de representación del comportamiento esperado de un algoritmo, permite a los estudiantes establecer un modelo estructural libre de “brincos” o “bifurcaciones” que aumentan, de forma artificial, la complejidad original del algoritmo.

Facilita el proceso de aprendizaje y migración de un lenguaje de programación a otros, ya que presenta los ejemplos y algoritmos en dos lenguajes de programación C y Java, permitiendo al estudiante una transición paulatina de la programación estructurada a la orientación a objetos, apoyándose en este proceso con el uso de los diagramas de UML.

Siendo un tema primordial para la resolución de algoritmos el uso de estructuras de datos, el autor dedica varios capítulos a la discusión de estas, iniciando con las estructuras de datos más simples, pasando por operaciones sobre archivos y terminando con una amplia explicación del uso de las estructuras de datos dinámicos lineales en Java.

El autor, adicionalmente, ofrece videos como complemento a la exposición de temas complejos, con el objetivo de disminuir la dificultad en su comprensión, los cuales pueden ser vistos en Internet. Este recurso permite el desarrollo del aprendizaje autodidacta del estudiante a su propio ritmo y, para el docente, puede ser un recurso didáctico que facilite su labor.

Por su organización y contenido, este libro puede ser utilizado como libro de texto siendo una guía en el proceso de enseñanza-aprendizaje o también, como libro de consulta para una o varias asignaturas ya que los contenidos pueden ser abordados de manera independiente.

El libro puede emplearse en asignaturas tales como Fundamentos de Programación, Estructuras de Datos y claro está, en la asignatura de Algoritmos Computacionales, esto dependerá de los contenidos del programa de estudio correspondiente.

Finalmente, a nuestro parecer, el libro despertará la curiosidad de los estudiantes y los animará a continuar por el camino de la construcción de sistemas de cómputo, por lo cual es posible que este libro sea su primer paso para ingresar al fascinante mundo de las Ciencias Computacionales.

Sergio Fuenlabrada Velázquez

Edna Martha Miranda Chávez

PROFESORES-INVESTIGADORES

INSTITUTO POLITÉCNICO NACIONAL

UNIDAD PROFESIONAL INTERDISCIPLINARIA DE INGENIERÍA

Y CIENCIAS SOCIALES Y ADMINISTRATIVAS

Cd. de México, mayo de 2012

Introducción

Algoritmos a fondo es un libro de nivel universitario pensado para cubrir las necesidades de los alumnos que cursan las materias de Algoritmos I, II y III, teniendo en cuenta el aprendizaje basado en competencias.

La obra comienza desde “cero” explicando los conceptos de “algoritmo”, “estructuras de control”, “variables”, “lenguajes de programación”, etcétera; y llega hasta el análisis e implementación de algoritmos complejos como ser los diferentes algoritmos de ordenamiento (*quicksort*, *heapsort*, etc.), recorridos sobre árboles, grafos y recursión.

Se estudian estructuras de datos estáticas y dinámicas, lineales y no lineales. Comenzando con programación estructurada e implementaciones en lenguaje C, hasta llegar a la programación orientada a objetos con implementaciones en lenguaje Java, previo paso por C++.

El libro se desarrolla como un “curso de programación” donde se guía al alumno en un proceso de aprendizaje durante el cual podrá adquirir la lógica necesaria para diseñar e implementar algoritmos.

Cada capítulo introduce un mayor nivel de dificultad, ya sea incorporando nuevos conceptos y recursos, o bien profundizando en técnicas de programación más complejas.

El autor remarca la idea de “curso de programación” ya que esta es totalmente opuesta al concepto de “libro tradicional” que, generalmente, presenta una estructura mucho más rígida, tipo “diccionario” o “enciclopedia”.

Cualquier alumno universitario debería poder leer el libro y aprender a programar por sí solo sin experimentar ningún tipo de dificultad ya que este es uno de los principales objetivos del libro.

La obra se complementa con una serie tutoriales grabados en video en los que el autor explica temas que dada su naturaleza resultarían extremadamente tediosos de leer.

Algoritmos a fondo se compone de cuatro módulos que agrupan los diferentes capítulos, según el siguiente criterio:

El Módulo 1 “Programación estructurada”, comienza desde “cero” y llega hasta el estudio de “estructuras dinámicas lineales”, pasando por “administración de archivos”, *arrays*, TADs, etcétera.

La implementación de los algoritmos y conceptos que aquí se estudian se basa en los diagramas de Chapín; la estructura de estos diagramas es mucho más rígida que la del diagrama de flujo tradicional y ayuda al alumno a pensar en algoritmos compuestos por bloques de “única entrada” y “única salida”. Este razonamiento constituye una de las premisas fundamentales de la programación estructurada.

Todos los algoritmos y programas que se exponen en el libro cuentan con su correspondiente codificación en lenguaje C, documentada y explicada.

Dentro del módulo 1, se incluyen capítulos que explican el lenguaje programación C que, como sabemos, es suficientemente complejo por sí mismo. Principalmente, el manejo cadenas de caracteres y los conceptos de “dirección de” y “contenido de” para poder pasarle argumentos por referencia a las funciones.

Demás está decir que el concepto de modularización es fundamental. Tanto es así que se estudia a partir del Capítulo 3 y se aplica de ahí en adelante.

En el Módulo 2 “Programación orientada a objetos”, se explican los conceptos de programación orientada a objetos, comenzando por la idea de “encapsulamiento”. Es decir: diseñar clases cuyos métodos encapsulen algoritmos complejos de forma tal un programador con menos conocimientos o menos experiencia los pueda utilizar sin tener que preocuparse por comprender su implementación.

Durante este módulo se realiza la transición entre los lenguajes C y Java; para amortiguar este proceso se estudia también algo de código C++.

En C++ se explican los conceptos de “clase y objeto”, encapsulamiento y generalizaciones mediante el uso de *templates*.

Los temas fuertes de la programación orientada a objetos como ser polimorfismo, *interfaces*, factorías, colecciones y clases genéricas se tratan directamente en Java.

El Módulo 3 “Aplicación práctica” es, en sí mismo, un ejercicio integrador cuyo desarrollo requerirá aplicar gran parte de los conocimientos adquiridos durante los dos módulos anteriores. Estamos hablando de un programa compresor/descompresor de archivos basado en el “algoritmo de Huffman”. Aquí se obtendrá suficiente evidencia de las competencias adquiridas hasta el momento.

Esta aplicación constituye un excelente ejercicio que inducirá al alumno a aplicar los principales conceptos estudiados desde el comienzo del libro: *arrays*, archivos y listas.

El algoritmo utiliza un árbol binario para generar los códigos Huffman que reemplazarán a los *bytes* de la fuente de información original. El hecho de que el tema “árbol binario” aún no haya sido estudiado representa una excelente oportunidad para aplicar los conceptos de abstracción y encapsulamiento expuestos en los capítulos del Módulo 2 sobre programación orientada a objetos.

El Módulo 3 se complementa con un *setup* que incluye clases utilitarias con las que el alumno podrá recorrer un árbol binario, leer y escribir “bit por bit” en un archivo, etcétera.

En el Módulo 4 “Conceptos avanzados”, se estudian algoritmos y estructuras de datos que revisten mayor nivel de complejidad.

Comenzando por el tema de recursión, se comparan las implementaciones recursivas e iterativas de diferentes funciones. Por ejemplo, el caso típico de la función *factorial* y el caso extremo de la función de *Fibonacci* cuya versión recursiva es incapaz de resolver, en un tiempo razonable, los términos de la serie superiores a 50.

Se analizan estructuras de datos no lineales: árboles y grafos; siempre aplicándolas a la solución de casos cotidianos para captar el interés del lector. Por ejemplo, una estructura de datos y un algoritmo que permitan implementar un *autosuggest* como el que utiliza Google en su buscador.

Durante los diferentes capítulos de este módulo, se explican algoritmos complejos como ser los diferentes métodos de ordenamiento, implementaciones alternativas mediante “programación dinámica” y los algoritmos tradicionales que operan sobre grafos: Dijkstra, Prim y Kruskal.

El docente debe saber que, en cada capítulo, se mencionan las competencias específicas a desarrollar y que en la página Web del libro dispone de una guía detallada de las competencias que se desarrollan a lo largo del libro y las evidencias que se pueden recolectar.

Introducción a los algoritmos
y a la programación de computadoras

Contenido

1.1 Introducción.....2

1.2 Concepto de algoritmo2

1.3 Conceptos de programación5

1.4 Representación gráfica de algoritmos.....7

1.5 Nuestro primer programa11

1.6 La memoria de la computadora14

1.7 Las variables17

1.8 Operadores aritméticos22

1.9 Expresiones lógicas29

1.10 Operadores de bits30

1.11 Resumen.....33

1.12 Contenido de la página Web de apoyo33

Objetivos del capítulo

- Entender los conceptos básicos sobre algoritmos y programación.
- Identificar los principales recursos lógicos y físicos que utilizan los programas.
- Estudiar el teorema de la programación estructurada y las estructuras de control que este teorema describe.
- Conocer las herramientas imprescindibles de programación: lenguaje de programación, compilador, entorno de desarrollo (IDE), etcétera.
- Desarrollar, compilar y ejecutar nuestro primer programa de computación.

Competencias específicas

- Dominar los conceptos básicos de la programación.
- Analizar problemas y representar su solución mediante algoritmos.
- Conocer las características principales del lenguaje C.
- Codificar algoritmos en el lenguaje de programación C.
- Compilar y ejecutar programas.

1.1 Introducción

Este capítulo introduce al lector en los conceptos iniciales sobre algoritmos y programación. Aquellos lectores que nunca han programado encontrarán aquí los conocimientos básicos para hacerlo.

Los contenidos son netamente introductorios ya que pretenden brindarle al lector las pautas, las expresiones y la jerga que necesitará conocer para poder leer y comprender los capítulos sucesivos.

Por lo anterior, la profundidad con la que se tratan los temas aquí expuestos es de un nivel inicial, ya que cada uno de estos temas será analizado en detalle llegado el momento, en los capítulos que así lo requieran.

1.2 Concepto de algoritmo

1.2.1 Definición de algoritmo y problema

Llamamos “algoritmo” al conjunto finito y ordenado de acciones con las que podemos resolver un determinado problema.

Llamamos “problema” a una situación que se nos presenta y que, mediante la aplicación de un algoritmo, pretendemos resolver.

Los algoritmos están presentes en nuestra vida cotidiana y, aún sin saberlo, aplicamos algoritmos cada vez que se nos presenta un problema sin importar cuál sea su grado de complejidad.

Para ejemplificar esto imaginemos que estamos dando un paseo por la ciudad y que, al llegar a una esquina, tenemos que cruzar la calle. Intuitivamente, aplicaremos el siguiente algoritmo:

- Esperar a que la luz del semáforo peatonal esté en verde (`esperarSemaforo`);
- Cruzar la calle (`cruzarCalle`);

Este algoritmo está compuesto por las acciones: `esperarSemaforo` y `cruzarCalle`, en ese orden, y es extremadamente simple gracias a que cada una de estas engloba a otro conjunto de acciones más puntuales y específicas. Por ejemplo:

La acción `esperarSemaforo` implica:

- Observar la luz del semáforo (`observarLuz`);
- **Si** la luz está en “rojo” o en “verde intermitente” **entonces**
 - esperar un momento (`esperar`);
 - **ir a:** `observarLuz`;

Por otro lado, la acción `cruzarCalle` implica:

- Bajar la calzada (`bajarCalzada`);
- Avanzar un paso (`avanzarPaso`);
- **Si** la distancia hacia la otra calzada es mayor que la distancia que podemos avanzar dando un nuevo paso **entonces**
 - **ir a:** `avanzarPaso`;
- Subir la calzada de la vereda de enfrente (`subirCalzada`);

Como vemos, cada acción puede descomponerse en acciones más puntuales y específicas. Por lo tanto, cada una de las acciones del algoritmo se resuelve con su propio algoritmo o secuencia finita y ordenada de acciones.

Cuando una acción se descompone en acciones más puntuales y específicas decimos que es un “módulo”.

Podemos seguir profundizando cada acción tanto como queramos. Por ejemplo, analicemos la acción (o el módulo) `bajarCalzada`: esta acción se resuelve:

- levantando un pie,
- adelantándolo,
- inclinando el cuerpo hacia adelante,
- y apoyando el pie en la calle,
- luego levantando el otro pie,
- adelantándolo y
- apoyándolo en la calle.

Claro que “levantar un pie” implica tensionar un conjunto de músculos, etcétera.

**Módulo**

Cuando una acción se descompone en acciones más puntuales y específicas la llamamos “módulo”.

1.2.2 Análisis del enunciado de un problema

Resulta evidente que, si vamos a diseñar un algoritmo para resolver un determinado problema, tenemos que tener totalmente estudiado y analizado el contexto de dicho problema. Esto implica:

- Comprender el alcance.
- Identificar los datos de entrada.
- Identificar los datos de salida o resultados.

El análisis anterior es fundamental para poder diseñar una estrategia de solución que será la piedra fundamental para el desarrollo del algoritmo.

1.2.2.1 Análisis del problema

Repasemos el contexto del problema de “cruzar la calle” que formulamos más arriba:

Caminando por la ciudad llegamos a una esquina y pretendemos cruzar la calle. Para poder cruzar con la mayor seguridad posible, tenemos que esperar que el semáforo habilite el paso peatonal. Hasta que esto ocurra nos encontraremos detenidos al lado del semáforo y una vez que este lo permita avanzaremos, paso a paso, hasta llegar a la vereda de enfrente. Se considera que el semáforo habilita el paso peatonal cuando emite una luz “verde fija”. En cambio si el semáforo emite una luz “roja” o “verde intermitente” se recomienda esperar.

1.2.2.2 Datos de entrada

En este contexto podemos identificar los siguientes datos de entrada:

- Nuestra posición inicial - posición en donde nos detuvimos a esperar a que el semáforo habilite el paso peatonal. La llamaremos `posInicial`.
- Luz del semáforo - el color de la luz que el semáforo está emitiendo. Lo llamaremos `luz`.
- Distancia de avance de paso - que distancia avanzamos cada vez que damos un nuevo paso. La llamaremos `distPaso`.
- Posición de la vereda de enfrente. La llamaremos `posFinal` ya que es allí hacia donde nos dirigimos.

1.2.2.3 Datos de salida

Si bien en este problema no se identifican datos de salida resulta evidente que luego de ejecutar la secuencia de acciones del algoritmo nuestra posición actual deberá ser la misma que la posición de la vereda de enfrente. Es decir, si llamamos `p` a nuestra posición actual resultará que su valor al inicio del algoritmo será `posInicial` y al final del mismo deberá ser `posFinal`.

1.2.3 Memoria y operaciones aritméticas y lógicas

En la vida real, cuando observamos la luz del semáforo almacenamos este dato en algún lugar de nuestra memoria para poder evaluar si corresponde cruzar la calle o no.

Generalmente, los datos de entrada ingresan al algoritmo a través de una acción que llamamos “lectura” o “ingreso de datos” y permanecen en la memoria el tiempo durante el cual el algoritmo se está ejecutando.

Por otro lado, el hecho de evaluar si corresponde o no cruzar la calle implica realizar una operación lógica. Esto es: determinar si la proposición “el semáforo emite luz roja o verde intermitente” resulta ser verdadera o falsa.

También realizaremos una operación lógica cuando estemos cruzando la calle y tengamos que determinar si corresponde dar un nuevo paso o no. Recordemos que llamamos `p` a nuestra posición actual, `posFinal` a la posición de la vereda de enfrente y `distPaso` a la distancia que avanzamos dando un nuevo paso. Por lo tanto, corresponde dar otro paso si se verifica que `posFinal > p+distPaso`. Aquí además de la operación lógica estamos realizando una operación aritmética: la suma `p+distPaso`.

En resumen, estos son los recursos que tenemos disponibles para diseñar y desarrollar algoritmos:

- La memoria.
- La posibilidad de realizar operaciones aritméticas.
- La posibilidad de realizar operaciones lógicas.

1.2.4 Teorema de la programación estructurada

Este teorema establece que todo algoritmo puede resolverse mediante el uso de tres estructuras básicas llamadas “estructuras de control”:

- La “estructura secuencial” o “acción simple”.
- La “estructura de decisión” o “acción condicional”.
- La “estructura iterativa” o “acción de repetición”.

Dos de estas estructuras las hemos utilizado en nuestro ejemplo de cruzar la calle. La estructura secuencial es la más sencilla y simplemente implica ejecutar una acción, luego otra y así sucesivamente.

La estructura de decisión la utilizamos para evaluar si correspondía dar un nuevo paso en función de nuestra ubicación actual y la ubicación de la vereda de enfrente.

Sin embargo, para resolver el problema hemos utilizado una estructura tabú: la estructura “**ir a**” o, en inglés, “*go to*” o “*goto*”. Esta estructura quedó descartada luego de que el teorema de la programación estructurada demostrara que con una adecuada combinación de las tres estructuras de control antes mencionadas es posible resolver cualquier algoritmo sin tener que recurrir al “*goto*” (o estructura “**ir a**”).

Para ejemplificarlo vamos a replantear el desarrollo de los algoritmos anteriores y reemplazaremos la estructura “**ir a**” (*goto*) por una estructura iterativa: la estructura “**repetir mientras que**”.



A Corrado Böhm y Giuseppe Jacopini se les atribuye el teorema de la programación estructurada, debido a un artículo de 1966.

David Harel rastreó los orígenes de la programación estructurada hasta la descripción de 1946 de la arquitectura de von Neumann y el teorema de la forma normal de Kleene.

La carta escrita en 1968 por Dijkstra, titulada “*Go To Considered Harmful*” reavivó el debate.

En la Web de apoyo, encontrará el vínculo a la página Web personal de Corrado Böhm.

Veamos el desarrollo del módulo `esperarSemaforo`:

Con “ ir a ”	Sin “ ir a ” (solución correcta)
• <code>observarLuz;</code> • Si la luz está en “rojo” o en “verde intermitente” entonces - <code>esperar;</code> - ir a: <code>observarLuz;</code>	• <code>observarLuz;</code> • Repetir mientras que la luz esté en “rojo” o en “verde intermitente” hacer - <code>esperar;</code> - <code>observarLuz;</code>

Veamos ahora el desarrollo del módulo `cruzarCalle`:

Con “ ir a ”	Sin “ ir a ” (solución correcta)
• <code>bajarCalzada;</code> • <code>avanzarPaso;</code> • Si la distancia hacia la otra calzada es mayor que la distancia que podemos avanzar dando un nuevo paso entonces - ir a: <code>avanzarPaso;</code> • <code>subirCalzada;</code>	• <code>bajarCalzada;</code> • <code>avanzarPaso;</code> • Repetir mientras que la distancia hacia la otra calzada sea mayor que la distancia que podemos avanzar dando un nuevo paso hacer - <code>avanzarPaso;</code> • <code>subirCalzada;</code>

Como vemos, la combinación “acción condicional + *goto*” se puede reemplazar por una estructura de repetición. Si bien ambos desarrollos son equivalentes la nueva versión es mejor porque evita el uso del *goto*, estructura que quedó en desuso porque trae grandes problemas de mantenibilidad.

1.3 Conceptos de programación

En general, estudiamos algoritmos para aplicarlos a la resolución de problemas mediante el uso de la computadora. Las computadoras tienen memoria y tienen la capacidad de resolver operaciones aritméticas y lógicas. Por lo tanto, son la herramienta fundamental para ejecutar los algoritmos que vamos a desarrollar.

Para que una computadora pueda ejecutar las acciones que componen un algoritmo tendremos que especificarlas o describirlas de forma tal que las pueda comprender.

Todos escuchamos alguna vez que “las computadoras solo entienden 1 (unos) y 0 (ceros)” y, efectivamente, esto es así, pero para nosotros (simples humanos) especificar las acciones de nuestros algoritmos como diferentes combinaciones de unos y ceros sería una tarea verdaderamente difícil. Afortunadamente, existen los lenguajes de programación que proveen una solución a este problema.

1.3.1 Lenguajes de programación

Las computadoras entienden el lenguaje binario (unos y ceros) y nosotros, los humanos, entendemos lenguajes naturales (español, inglés, portugués, etc.).

Los lenguajes de programación son lenguajes formales que se componen de un conjunto de palabras, generalmente en inglés, y reglas sintácticas y semánticas.

Podemos utilizar un lenguaje de programación para escribir o codificar nuestro algoritmo y luego, con un programa especial llamado “compilador”, podremos generar los “unos y ceros” que representan sus acciones. De esta manera, la computadora será capaz de comprender y convertir al algoritmo en un programa de computación.



Los lenguajes naturales son los que hablamos y escribimos los seres humanos: inglés, español, italiano, etc. Son dinámicos ya que, constantemente, incorporan nuevas variaciones, palabras y significados. Por el contrario, los lenguajes formales son rígidos y se construyen a partir de un conjunto de símbolos (alfabeto) unidos por un conjunto de reglas (gramática). Los lenguajes de programación son lenguajes formales.



El lenguaje C++ fue creado a mediados de los años ochenta por Bjarne Stroustrup, con el objetivo de extender al lenguaje de programación C con mecanismos que permitan la manipulación de objetos.



Java es un lenguaje de programación orientado a objetos, creado en la década del noventa por Sun Microsystems (actualmente adquirida por Oracle). Utiliza gran parte de la sintaxis de C y C++, pero su modelo de objetos es más simple.



El lenguaje de programación C fue creado por Dennis Ritchie (1941-2011). En 1967 Ritchie comenzó a trabajar para los laboratorios Bell, donde se ocupó, entre otros, del desarrollo del lenguaje B. Creó el lenguaje de programación C en 1972, junto con Ken Thompson. Ritchie también participó en el desarrollo del sistema operativo Unix.

Existen muchos lenguajes de programación: Pascal, C, Java, COBOL, Basic, Smalltalk, etc., y también existen muchos lenguajes derivados de los anteriores: Delphi (derivado de Pascal), C++ (derivado de C), C# (derivado de C++), Visual Basic (derivado de Basic), etcétera.

En este libro utilizaremos el lenguaje de programación C y, para los capítulos de encapsulamiento y programación orientada a objetos, C++ y Java.

1.3.2 Codificación de un algoritmo

Cuando escribimos las acciones de un algoritmo en algún lenguaje de programación decimos que lo estamos “codificando”. Generalmente, cada acción se codifica en una línea de código.

Al conjunto de líneas de código, que obtenemos luego de codificar el algoritmo, lo llamamos “código fuente”.

El código fuente debe estar contenido en un archivo de texto cuyo nombre debe tener una extensión determinada que dependerá del lenguaje de programación que hayamos utilizado. Por ejemplo, si codificamos en Pascal entonces el nombre del archivo debe tener la extensión “.pas”. Si codificamos en Java entonces la extensión del nombre del archivo deberá ser “.java” y si el algoritmo fue codificado en C entonces el nombre del archivo deberá tener la extensión “.c”.

1.3.3 Bibliotecas de funciones

Los lenguajes de programación proveen bibliotecas o conjuntos de funciones a través de las cuales se ofrece cierta funcionalidad básica que permite, por ejemplo, leer y escribir datos sobre cualquier dispositivo de entrada/salida como podría ser la consola.

Es decir, gracias a que los lenguajes proveen estos conjuntos de funciones los programadores estamos exentos de programarlas y simplemente nos limitaremos a utilizarlas.

Programando en C, por ejemplo, cuando necesitamos mostrar un mensaje en la pantalla utilizaremos la función `printf` y cuando queramos leer datos a través del teclado utilizaremos la función `scanf`. Ambas funciones forman parte de la biblioteca estándar de entrada/salida de C, también conocida como “stdio.h”.

1.3.4 Programas de computación

Un programa es un algoritmo que ha sido codificado y compilado y que, por lo tanto, puede ser ejecutado en una computadora.

El algoritmo constituye la lógica del programa. El programa se limita a ejecutar cada una de las acciones del algoritmo. Por esto, si el algoritmo tiene algún error entonces el programa también lo tendrá y si el algoritmo es lógicamente perfecto entonces el programa también lo será.

El proceso para crear un nuevo programa es el siguiente:

- Diseñar y desarrollar su algoritmo.
- Codificar el algoritmo utilizando un lenguaje de programación.
- Compilarlo para obtener el código de máquina o archivo ejecutable (los “unos y ceros”).

Dado que el algoritmo es la parte lógica del programa muchas veces utilizaremos ambos términos como sinónimos ya que para hacer un programa primero necesitaremos diseñar su algoritmo. Por otro lado, si desarrollamos un algoritmo seguramente será para, luego, codificarlo y compilarlo, es decir, programarlo.

1.3.5 Consola

Llamamos “consola” al conjunto compuesto por el teclado y la pantalla de la computadora en modo texto. Cuando hablemos de ingreso de datos por consola nos estaremos refiriendo al teclado y cuando hablemos de mostrar datos por consola estaremos hablando de la pantalla, siempre en modo texto.

1.3.6 Entrada y salida de datos

Llamamos “entrada” al conjunto de datos externos que ingresan al algoritmo. Por ejemplo, el ingreso de datos por teclado, la información que se lee a través de algún dispositivo como podría ser un lector de código de barras, un *scanner* de huellas digitales, etcétera.

Llamamos “salida” a la información que el algoritmo emite sobre algún dispositivo como ser la consola, una impresora, un archivo, etcétera.

La consola es el dispositivo de entrada y salida por omisión. Es decir, si hablamos de ingreso de datos y no especificamos nada más será porque el ingreso de datos lo haremos a través del teclado. Análogamente, si hablamos de mostrar cierta información y no especificamos más detalles nos estaremos refiriendo a mostrarla por la pantalla de la computadora, en modo texto.

1.3.7 Lenguajes algorítmicos

Llamamos “lenguaje algorítmico” a todo recurso que permita describir con mayor o menor nivel de detalle los pasos que componen un algoritmo.

Los lenguajes de programación, por ejemplo, son lenguajes algorítmicos ya que, como veremos más adelante, la codificación del algoritmo es en sí misma una descripción detallada de los pasos que lo componen. Sin embargo, para describir un algoritmo no siempre será necesario llegar al nivel de detalle que implica codificarlo. Una opción válida es utilizar un “pseudocódigo”.

1.3.8 Pseudocódigo

El pseudocódigo surge de mezclar un lenguaje natural (por ejemplo, el español) con ciertas convenciones sintácticas y semánticas propias de un lenguaje de programación. Justamente, el algoritmo que resuelve el problema de “cruzar la calle” fue desarrollado utilizando un pseudocódigo.

Los diagramas también permiten detallar los pasos que componen un algoritmo; por lo tanto, son lenguajes algorítmicos. En este libro profundizaremos en el uso de diagramas para luego codificarlos con el lenguaje de programación C.

1.4 Representación gráfica de algoritmos

Los algoritmos pueden representarse mediante el uso de diagramas. Los diagramas proveen una visión simplificada de la lógica del algoritmo y son una herramienta importantísima que utilizaremos para analizar y documentar los algoritmos o programas que vamos a desarrollar.

En este libro, utilizaremos una versión modificada de los diagramas de *Nassi-Shneiderman*, también conocidos como diagramas *Chapin*. Estos diagramas se componen de un conjunto de símbolos que permiten representar cada una de las estructuras de control que describe el teorema de la programación estructurada: la estructura secuencial, la estructura de decisión y la estructura de repetición.



Los diagramas Nassi-Shneiderman, también conocidos como “diagramas de Chapin”, fueron publicados en 1973 por Ben Shneiderman e Isaac Nassi en el artículo llamado “A short history of structured flowcharts”, con el objetivo de eliminar las líneas de los diagramas tradicionales y así reforzar la idea de estructuras de “única entrada y única salida”.

Si en la programación estructurada se elimina la sentencia GOTO entonces se deben eliminar las líneas en los diagramas que la representan.

1.4.1 Representación gráfica de la estructura secuencial o acción simple

La estructura secuencial se representa en un rectángulo o, si se trata de un ingreso o egreso de datos se utiliza un trapecio como observamos a continuación:

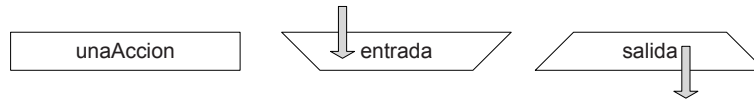


Fig. 1.1 Representación de estructuras secuenciales.

Nota: las flechas grises no son parte de los símbolos de entrada y salida, simplemente son ilustrativas.

1.4.2 Representación gráfica de la estructura de decisión

Esta estructura decide entre ejecutar un conjunto de acciones u otro en función de que se cumpla o no una determinada condición o expresión lógica.

Informalmente, diremos que una “expresión lógica” es un enunciado susceptible de ser verdadero o falso. Por ejemplo, “2 es par” o “5 es mayor que 8”. Ambas expresiones tienen valor de verdad, la primera es verdadera y la segunda es falsa.

La estructura se representa dentro de una “casa con dos habitaciones y techo a dos aguas”. En “el altillo” indicamos la expresión lógica que la estructura debe evaluar. Si esta expresión resulta ser verdadera entonces se ejecutarán las acciones ubicadas en la sección izquierda. En cambio, si la expresión resulta ser falsa se ejecutarán las acciones que estén ubicadas en la sección derecha.

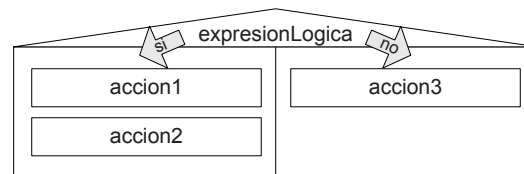


Fig. 1.2 Representación gráfica de la estructura condicional.

En este caso, si se verifica `expresionLogica` se ejecutarán las acciones `accion1` y `accion2`. En cambio, si `expresionLogica` resulta ser falsa se ejecutará únicamente la acción `accion3`. Las flechas grises son ilustrativas y no son parte del diagrama.

1.4.3 Representación gráfica de la estructura de repetición

La estructura de repetición se representa en una “caja” con una cabecera que indica la expresión lógica que se debe cumplir para seguir ejecutando las acciones contenidas en la sección principal.

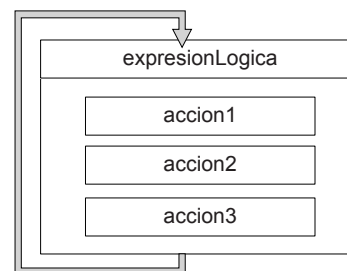


Fig. 1.3 Representación gráfica de la estructura de repetición.

En este caso, mientras `expresionLogica` resulte ser verdadera se repetirán una y otra vez las acciones `accion1`, `accion2` y `accion3`. Por lo tanto, dentro de alguna de estas acciones deberá suceder algo que haga que la condición del ciclo se deje de cumplir. De lo contrario, el algoritmo se quedará iterando dentro de este ciclo de repeticiones.

Con lo estudiado hasta aquí, podemos representar gráficamente el algoritmo que resuelve el problema de cruzar la calle.

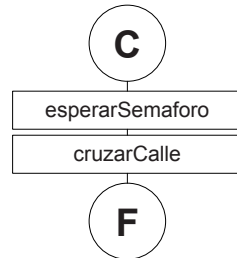


Fig. 1.4 Diagrama del algoritmo principal.

El diagrama principal comienza con una **C** y finaliza con una **F**, iniciales de “Comienzo” y “Fin”, cada una encerrada dentro de un círculo. Luego, las acciones simples `esperarSemaforo` y `cruzarCalle` se representan dentro de rectángulos, una detrás de la otra. Como cada una de estas acciones corresponde a un módulo tendremos que proveer también sus propios diagramas.

1.4.4 Representación gráfica de módulos o funciones

Como estudiamos anteriormente, un módulo representa un algoritmo que resuelve un problema específico y puntual. Para representar, gráficamente, las acciones que componen a un módulo, utilizaremos un paralelogramo con el nombre del módulo como encabezado y una **R** (inicial de “Retorno”) encerrada dentro de un círculo para indicar que el módulo finalizó y que se retornará el control al algoritmo o programa principal.

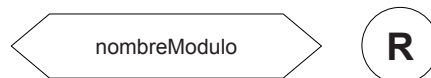


Fig. 1.5 Encabezado y finalización de un módulo.

Con esto, podemos desarrollar los diagramas de los módulos `esperarSemaforo` y `cruzarCalle`.

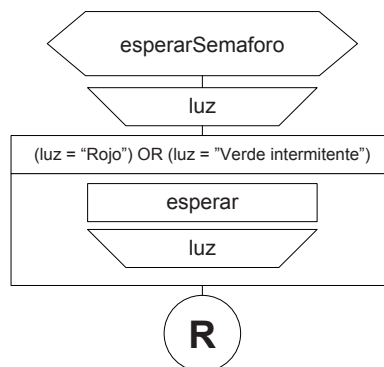


Fig. 1.6 Módulo `esperarSemaforo`.

En el diagrama del módulo `esperarSemaforo`, leemos el color de la luz que actualmente está emitiendo el semáforo y lo “mantenemos en memoria” asociándolo al identificador `luz`. Esto lo hacemos indicando el nombre del identificador dentro del símbolo de lectura o ingreso de datos.

Luego ingresamos a una estructura de repetición que iterará mientras que la expresión lógica `(luz = "Rojo") OR (luz = "Verde intermitente")` resulte verdadera. Las acciones encerradas dentro de esta estructura se repetirán una y otra vez mientras la condición anterior continúe verificándose. Estas acciones son: `esperar` y volver a leer la luz que emite el semáforo.

Evidentemente, en algún momento, el semáforo emitirá la luz “verde fija”, la condición de la cabecera ya no se verificará y el ciclo de repeticiones dejará de iterar.

Veamos ahora el diagrama del módulo `cruzarCalle`.

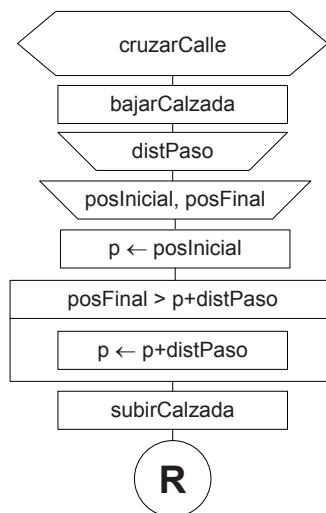


Fig. 1.7 Módulo `cruzarCalle`.

El análisis de este módulo es similar al anterior. Aquí primero invocamos al módulo `bajarCalzada` y luego, ingresamos los datos `distPaso` (distancia que avanzamos al dar un nuevo paso), `posInicial` y `posFinal` (la posición donde estamos parados y la posición de la vereda de enfrente respectivamente). A continuación, “asignamos” al identificador `p` el valor de `posInicial` y luego, ingresamos a una estructura de repetición que iterará mientras que se verifique la expresión lógica:

`posFinal > p+distPaso`

Dentro de la estructura de repetición, tenemos que avanzar un paso. Si bien en el primer análisis del algoritmo habíamos definido un módulo `avanzarPaso`, en este caso, preferí o reemplazarlo directamente por la acción:

`p ← p+distPaso`

Fig. 1.8 Asignación y acumulador.

Esta acción indica que a `p` (identificador que contiene nuestra posición) le asignamos la suma de su valor actual más el valor de `distPaso` (distancia que avanzamos dando un nuevo paso). Luego de esto estaremos más cerca de `posFinal`.

Para finalizar invocamos al módulo `subirCalzada` y retornamos al diagrama del programa principal.

1.5 Nuestro primer programa

Vamos a analizar, diseñar y programar nuestro primer algoritmo. Se trata de un programa trivial que simplemente emite en la consola la frase “Hola Mundo”.

La representación gráfica de este algoritmo es la siguiente:

El algoritmo comienza, emite un mensaje en la consola diciendo “Hola Mundo” y finaliza.

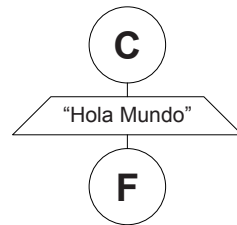


Fig. 1.9 Representación gráfica del algoritmo “Hola Mundo”.

1.5.1 Codificación del algoritmo utilizando el lenguaje C

El siguiente paso es codificar el algoritmo. Con el diagrama resuelto, la codificación se limita a transcribirlo en el lenguaje de programación elegido que, en este caso, es C.

Veamos el código fuente y luego lo analizaremos:

```
#include <stdio.h>

int main()
{
    printf("Hola Mundo\n");
    return 0;
}
```

Todos los programas C se codifican dentro de la función `main`. La palabra “*main*” (que significa “principal”) es una palabra reservada y se utiliza como encabezado del programa principal.

Para emitir el mensaje “Hola Mundo” en la consola, utilizamos la función `printf`. Como comentamos anteriormente esta función es parte de la biblioteca estándar de entrada/salida de C llamada “`stdio.h`”.

La función `printf` recibe como argumento una cadena de caracteres y la imprime en la consola. Notemos que al texto que queremos imprimir le agregamos el carácter especial `\n` (léase “barra ene”). Este carácter representa un salto de línea.

Prácticamente, todos los programas C comienzan con la directiva (de preprocesador):

```
#include <stdio.h>
```

Esta directiva “incluye” las definiciones de las funciones de entrada y salida de la biblioteca estándar.



El preprocesador es un programa que es invocado por el compilador antes de comenzar su trabajo para, por ejemplo, eliminar los comentarios y otros elementos no necesarios para la compilación.



El valor de retorno de la función `main`, también llamado *exit code*, indica como finalizó el programa. Un valor igual a 0 representa una finalización exitosa mientras que un valor diferente indica algún grado de anormalidad.

El *exit code* se utiliza en la programación de *scripts* ya que en función de este se puede determinar si el programa concretó su tarea o no.

Los bloques de código se delimitan entre `{` y `}` (“llave de apertura” y “llave de cierre”). En este caso, el único bloque de código es el bloque que delimita las líneas que forman parte de la función `main` o programa principal.

La sintaxis del lenguaje de programación C exige que todas las líneas de código (salvo las directivas de preprocesador como `include` y los delimitadores de bloques) finalicen con `;` (punto y coma).

Por último, con la sentencia `return` hacemos que la función retorne (o devuelva) un valor a quién la invocó. Este valor es conocido como “el valor de retorno de la función” y lo analizaremos en detalle en el Capítulo 3.

1.5.2 El archivo de código fuente

La codificación del algoritmo constituye el “código fuente”. Estas líneas de código deben estar contenidas en un archivo de texto cuyo nombre tiene que tener la extensión “.c”. En este caso, el nombre del archivo de código fuente podría ser: `HolaMundo.c`.

1.5.3 Comentarios en el código fuente

A medida que la complejidad del algoritmo se incrementa, el código fuente que tendremos que escribir para codificarlo será más complejo y, por lo tanto, más difícil de entender. Por este motivo, los lenguajes de programación permiten agregar comentarios de forma tal que el programador pueda anotar acotaciones que ayuden a hacer más legible el programa que desarrolló.

En C podemos utilizar dos tipos de comentarios:

- Comentarios en línea: cualquier línea de código que comience con “doble barra” será considerada como un comentario.

```
// esto es un comentario en linea
```

- Comentarios en varias líneas: son aquellos que están encerrados entre un “barra asterisco” y un “asterisco barra” como se muestra a continuación:

```
/* Esto es un
comentario en varias
lineas de codigo */
```

A continuación, agregaremos algunos comentarios al programa “HolaMundo”.

```
/*
Programa: HolaMundo.c
Autor: Pablo Sznajdleder
Fecha: 24/junio/2012
*/

// incluye las definiciones de las funciones de la biblioteca estandar
#include <stdio.h>

// programa o funcion principal
int main()
{
    // escribe un mensaje en la consola
    printf("Hola Mundo\n");
    return 0;
}
```


Los comentarios dentro del código fuente ayudan a que el programa sea más legible para el programador y, obviamente, no son tomados en cuenta por el compilador.

1.5.4 La compilación y el programa ejecutable

El próximo paso será compilar el código fuente para obtener el programa ejecutable y poder ejecutarlo (correrlo) en la computadora.

La compilación se realiza con un programa especial llamado “compilador”. Existen diversos compiladores que permiten compilar programas escritos en C. En este libro utilizaremos el compilador GCC o MinGW (<http://www.mingw.org/>).

Para compilar el programa `HolaMundo.c` con GCC escribimos el siguiente comando en la línea de comandos:

```
gcc HolaMundo.c -o HolaMundo.exe
```

Luego de esto se generará, dentro de la misma carpeta en la que compilamos el programa, un nuevo archivo llamado `HolaMundo.exe` que al ejecutarlo mostrará el texto “Hola Mundo” en la consola.

1.5.5 El entorno integrado de desarrollo (IDE)

Cuando los algoritmos adquieren mayor nivel de complejidad, su codificación y compilación pueden convertirse en verdaderos problemas.

Una IDE (por sus siglas en inglés, *Integrated Development Environment*) es una herramienta que integra todos los recursos que necesita un programador para codificar, compilar, depurar, ejecutar y documentar sus programas.

Para programar en C existen diferentes IDEs. Algunas de estas son: Visual Studio (de Microsoft), C++ Builder (de Borland), Dev C (*open source*), Eclipse (*open source*), etcétera.

En este libro utilizaremos Eclipse. Su instalación, configuración y funcionamiento lo analizaremos en los videotutoriales que lo acompañan. Sin embargo, y a modo de ejemplo, a continuación haremos un breve repaso por la herramienta.



Instalación y uso de Eclipse para C

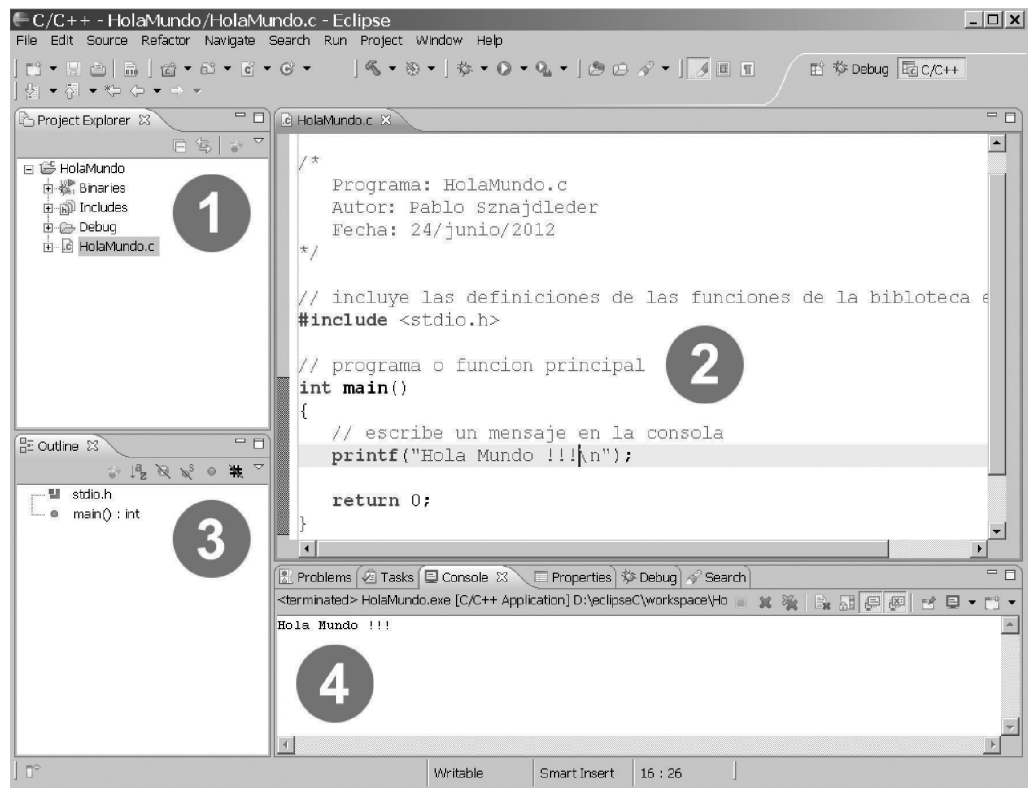


Fig. 1.10 El entorno integrado de desarrollo Eclipse.

En la imagen podemos identificar 4 secciones:

- El **Project Explorer** (1) – Aquí vemos todos los archivos de código fuente que integran nuestro proyecto. En este caso, el proyecto tiene un único archivo: `HolaMundo.c`.
- La **Ventana de Edición** (2) – Es el área principal en donde podemos ver y editar cada uno de los archivos de código fuente.
- El **Outline** (3) – Aquí se detalla la estructura del archivo de código fuente que estamos editando. Se muestra un resumen de todas sus funciones, variables, los `.h` que incluye, etcétera.
- La **Consola** (4) – Es el área en donde ingresaremos los datos a través del teclado y donde se mostrarán los resultados.

1.6 La memoria de la computadora

Más arriba hablamos de entrada y salida de datos. Los datos ingresan al algoritmo a través de cualquier dispositivo de entrada y es nuestra responsabilidad mantenerlos en memoria para tenerlos disponibles y, llegado el momento, poderlos utilizar.

Dependiendo del contenido del dato, necesitaremos utilizar una mayor o menor cantidad de memoria para almacenarlo. Por ejemplo, si el dato corresponde al nombre de una persona probablemente 20 caracteres sean más que suficiente. En cambio, si el dato corresponde a su dirección postal, seguramente, necesitemos utilizar una mayor cantidad de caracteres, quizás 150 o 200.

1.6.1 El byte

La memoria se mide en *bytes*. Un *byte* constituye la mínima unidad de información que podemos almacenar en la memoria. En la actualidad, las computadoras traen grandes cantidades de memoria expresadas en múltiplos del *byte*. Por ejemplo, 1 *gigabyte* representa 1024 *megabytes*. 1 *megabyte* representa 1024 *kilobytes* y un *kilobyte* representa 1024 *bytes*.

A su vez, un *byte* representa un conjunto de 8 bits (dígitos binarios). Por lo tanto, en un *byte* podemos almacenar un número binario de hasta 8 dígitos.

Obviamente, aunque la representación interna del número sea binaria, nosotros podemos pensarlo en base 10 como veremos a continuación.



Bit es el acrónimo de *binary digit*. (dígito binario). Corresponde a un dígito del sistema de numeración binaria que puede representar uno de dos valores, 0 o 1. Es la unidad mínima de información usada en informática.

1.6.2 Conversión numérica: de base 2 a base 10

Un número entero representado en base 2 puede ser fácilmente representado en base 10 realizando los siguientes pasos:

1. Considerar que cada dígito binario representa una potencia de 2, comenzando desde 2^0 y finalizando en 2^{n-1} siendo n la cantidad de dígitos binarios con los que el número está siendo representado.
2. El número en base 10 se obtiene sumando aquellas potencias de 2 cuyo dígito binario sea 1.

Analicemos un ejemplo:

Sea el número binario: 10110011 entonces:

1	0	1	1	0	0	1	1
2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
128	64	32	16	8	4	2	1

Vemos que cada dígito binario representa una potencia de 2, desde la potencia 0 hasta la potencia 7, comenzando desde la derecha.

Concretamente, la representación decimal del número binario 10110011 se obtiene sumando aquellas potencias de 2 cuyo dígito binario es 1.

En este caso será: $2^0 + 2^1 + 2^4 + 2^5 + 2^7 = 179$.

Según lo anterior, el mayor valor numérico que podremos almacenar en 1 *byte* de información será: 11111111 (8 bits todos en 1) y corresponde al número decimal 255 (que coincide con $2^8 - 1$).

1.6.3 Dimensionamiento de los datos

El análisis anterior es fundamental para comprender que si, por ejemplo, el dato que va a ingresar a nuestro algoritmo corresponde a la edad de una persona podremos almacenarlo sin problemas en 1 *byte* de memoria ya que, salvo Matusalén, nadie llega a vivir 255 años.

También podríamos almacenar en 1 *byte* datos tales como:

- El día del mes (un número que puede estar entre 1 y 31)
- La nota obtenida en un examen (número entre 1 y 10 o entre 1 y 100)
- El número que salió en la ruleta (número entre 0 y 36)



En el Antiguo Testamento, se menciona a Matusalén como la persona más anciana. Vivió hasta los 969 años según Génesis 5:27.

En cambio, no podemos almacenar en 1 *byte*: el día del año ya que este valor será un número entre 1 y 365. Y no sería prudente almacenar en 1 *byte* la distancia (expresada en kilómetros) que existe entre dos ciudades ya que muy probablemente esta distancia sea superior a los 255 km.

Para estos casos necesitaremos utilizar 2 *bytes* de memoria. En 16 bits podremos almacenar valores numéricos hasta $2^{16}-1 = 65535$.

1.6.4 Los números negativos

En el párrafo anterior, vimos que en 1 *byte* podemos representar valores numéricos enteros entre 0 y 255, en 2 *bytes* podemos representar valores numéricos enteros entre 0 y 65535 y, en general, en n *bytes* podremos representar valores numéricos enteros entre 0 y 2^n-1 .

Para representar valores numéricos enteros negativos, se reserva el bit más significativo del conjunto de *bytes* para considerarlo como “bit de signo”. Si este bit vale 1 entonces los restantes bits del conjunto de *bytes* representarán un valor numérico entero negativo. En cambio, si este bit vale 0 entonces los bits restantes estarán representando un valor numérico entero positivo.

Así, en un *byte* con bit de signo, el mayor valor que se podrá representar será 01111111 (un cero y siete unos) y corresponde al valor decimal 127 (es decir: 2^7-1) y el menor valor que se podrá almacenar será 10000000 (un 1 y siete 0) y corresponde al valor decimal: -127 (*).

Análogamente, en 2 *bytes* con bit de signo el mayor valor que podremos representar será: 0111111111111111 (un cero y 15 unos), que corresponde al valor decimal 32767 mientras que el menor valor será -32767 (*).

Ahora bien, si 00000000 representa al valor decimal 0 (cero) entonces ¿la combinación 10000000 representa a -0 (“menos cero”)?

La respuesta a este planteamiento se fundamenta en la representación binaria interna que utilizan las computadoras para los números negativos y que se basa en el complemento a 2 del número positivo. En este esquema los valores negativos se representan invirtiendo los bits que representan a los números positivos y luego, sumando 1 al resultado.

La operación inversa nos permite obtener el valor absoluto de cualquier número binario que comience con 1. Así, el valor absoluto del número 10000000 (-0) será: $01111111+1 = 10000000 = 128$.

Por este motivo, los tipos de datos que representan números enteros signados admiten valores entre -2^{n-1} y $+2^{n-1}-1$ siendo n la cantidad de bits utilizados en dicha representación. 1 *byte* implica $n=8$, 2 *bytes* implica $n=16$, etcétera.

1.6.5 Los caracteres

Los caracteres se representan como valores numéricos enteros positivos. Cada carácter tiene asignado un valor numérico definido en la tabla ASCII. En esta tabla se define que el carácter ‘A’ se representa con el valor 65, el carácter ‘B’ con el 66 y así sucesivamente. Para representar al carácter ‘a’ se utiliza el valor 97, el ‘b’ se representa con el valor 98, el carácter ‘0’ con el valor 48, el ‘1’ con el 49, etcétera.

Por lo tanto, para representar cada carácter alcanzará con 1 *byte* de memoria y n caracteres requerirán n *bytes* de memoria.

1.7 Las variables

Las variables representan un espacio de la memoria de la computadora. A través de una variable, podemos almacenar temporalmente datos para tenerlos disponibles durante la ejecución del programa.

Para utilizar una variable, tenemos que especificar un nombre y un tipo de datos. Al nombre de la variable lo llamamos “identificador”. En general, un identificador debe comenzar con una letra y no puede contener espacios en blanco, signos de puntuación u operadores.

Nombres de variables o identificadores **válidos** son:

- fecha
- fec
- fechaNacimiento
- fechaNac
- fec1
- fec2
- iFechaNac

Nombres de variables o identificadores **incorrectos** son:

- 2fecha (no puede comenzar con un número)
- -fecha (no puede comenzar con un signo “menos”)
- fecha nacimiento (no puede tener espacios en blanco)
- fecha-nacimiento (no puede tener el signo “menos”)
- fecha+nacimiento (no puede tener el signo “más”)

Los valores que mantienen las variables pueden cambiar durante la ejecución del programa, justamente por eso, son “variables”. Para que una variable adquiera un determinado valor se lo tendremos que asignar manualmente con el operador de asignación o bien leer un valor a través de algún dispositivo de entrada y almacenarlo en la variable. Esto lo representaremos gráficamente de la siguiente manera:

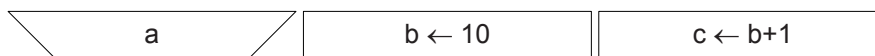


Fig. 1.11 Representación gráfica de lectura y asignación.

En las figuras vemos representadas, de izquierda a derecha, las siguientes acciones:

- Leo un valor por consola y lo asigno a la variable `a`.
- Asigno el valor 10 a la variable `b`.
- Asigno a la variable `c` el valor que contiene la variable `b` “más 1”.

Recordemos que para definir una variable es necesario especificar su tipo de datos lo que significa que una variable solo podrá contener datos del mismo tipo. Es decir, la variable puede tomar diferentes valores durante la ejecución del programa, pero todos estos valores siempre serán del mismo tipo: el tipo de datos con el que la variable fue definida.

1.7.1 Convención de nomenclatura para variables

Si bien el lenguaje de programación solo exige que los nombres de las variables o identificadores respeten las restricciones mencionadas más arriba, en este libro, adoptaremos la siguiente convención de nomenclatura:

Nombres simples: deben escribirse completamente en minúscula. Por ejemplo: `fecha`, `nombre`, `edad`, `direccion`, etcétera.

Nombres compuestos: si el nombre de la variable está compuesto por dos o más palabras entonces cada palabra, a excepción de la primera, debe comenzar en mayúscula. Por ejemplo: `fechaNacimiento`, `nombreYApellido`, etcétera.

Nunca el nombre de una variable debería comenzar en mayúscula ni escribirse completamente en mayúscula.

1.7.2 Los tipos de datos

Dependiendo del tipo de datos que definamos para una variable, esta estará representando una mayor o menor cantidad de *bytes* de memoria y, por lo tanto, nos permitirá almacenar mayor o menor cantidad de información.

Según su contenido, los datos pueden clasificarse como numéricos, alfanuméricos o lógicos.

Por ejemplo: la dirección postal de una persona es un dato **alfanumérico**. Este tipo de dato representa una sucesión de caracteres alfabéticos y/o numéricos como podría ser "Av. Del Libertador Nro. 2345, piso 6".

En cambio, la edad de una persona es un dato numérico, entero positivo y acotado ya que, como analizamos más arriba, 255 es una edad absurda a la que un ser humano nunca podrá llegar. Es decir que este dato es un número **entero corto** y sin bit de signo o *unsigned* ya que ninguna persona podrá tener una edad negativa.

El kilometraje de un auto, por ejemplo, es un valor entero positivo y, potencialmente, muy grande. No podrá representarse en 1 *byte* (255), ni siquiera en 2 *bytes* (65535). Tendremos que utilizar una mayor cantidad de *bytes*. A este tipo de datos lo llamaremos **entero largo**. Este dato también es *unsigned* ya que un auto no puede tener un kilometraje negativo.

La distancia (expresada en kilómetros) que existe entre dos ciudades es un dato que puede representarse en 2 *bytes*. Esto se desprende del siguiente razonamiento: Como el diámetro del ecuador es de aproximadamente 12710 km entonces el perímetro de la Tierra se puede calcular como $\pi \times \text{diámetro} = 40074$ km, también aproximado. Es decir que, en el peor de los casos, una ciudad puede estar en un punto de la Tierra y la otra puede estar justo al otro lado del mundo y aun así, la distancia nunca será mayor que la mitad de perímetro de la Tierra: 20037 km.

El saldo de una cuenta bancaria es un valor numérico real. Es decir, probablemente tenga decimales y podrá ser positivo o negativo. A este tipo de datos lo llamamos **flotante**. La representación interna de los números con punto flotante la analizaremos más adelante.

Según sea el grado de precisión que demande el valor real que tenemos que representar, su tipo podrá ser simplemente "flotante" o bien flotante de **doble precisión**.

Por último, los datos lógicos o **booleanos** son aquellos que tienen valor de verdad. Este valor puede ser verdadero o falso. En general, utilizamos este tipo de dato para almacenar el resultado de una operación lógica.

1.7.3 Los tipos de datos provistos por el lenguaje C

Los lenguajes de programación proveen tipos de datos con los cuales se puede definir (o declarar) variables.

Como vimos más arriba, para utilizar una variable será necesario definir su identificador (nombre de la variable) y su tipo de datos. En C disponemos de los siguientes tipos:



El ecuador divide al globo en el hemisferio norte y el hemisferio sur, es una línea imaginaria (un círculo máximo) que se encuentra, exactamente, a la misma distancia de los polos geográficos.

Naturaleza	Tipo	Bytes	Descripción
Tipos numéricos enteros	<i>short</i>	1	entero corto
	<i>int</i>	2	entero
	<i>long</i>	4	entero largo
	<i>char</i>	1	carácter (entero corto)
Tipos numéricos flotantes (o reales)	<i>float</i>	4	flotante
	<i>double</i>	8	flotante doble precisión

Nota muy importante: Las cantidades de *bytes* no siempre serán las mencionadas en la tabla ya que estas dependerán del compilador y de la arquitectura del *hardware* en donde estemos compilando y ejecutando nuestro programa. Sin embargo, en este libro, por cuestiones didácticas, consideraremos que esta será la cantidad de *bytes* de memoria reservada para cada tipo de dato.

A cada uno de los tipos de datos enteros se les puede aplicar el modificador `unsigned` con lo que podremos indicar si queremos o no que se deje sin efecto el bit de signo. Con esto, aumentamos el rango de valores positivos que admite el tipo de datos a costa de sacrificar la posibilidad de almacenar valores negativos.

Los datos lógicos o booleanos se manejan como tipos enteros, considerando que el valor 0 es “falso” y cualquier otro valor distinto de 0 es “verdadero”. En otros lenguajes, como Pascal o Java, se provee el tipo de datos `boolean`, pero en C los datos lógicos simplemente se trabajan como `int`.

Por último, los datos alfanuméricos o “cadenas de caracteres” se representan como “conjuntos de variables” de tipo `char`. A estos “conjuntos” se los denomina *arrays*, un tema que estudiaremos en detalle más adelante.

1.7.3.1 Notación húngara

La notación húngara es una convención de nomenclatura que propone anteponer al nombre de la variable un prefijo que, a simple vista, permita identificar su tipo de datos.

Por ejemplo: `iContador` (“i” de `int`), `sNombre` (“s” de `string`), `bFin` (“b” de `boolean`), `dPrecioVenta` (“d” de `double`), etcétera.

Si bien la notación húngara cuenta con una importante comunidad de detractores que aseguran que, a la larga, complica la legibilidad y la mantenibilidad del código fuente, personalmente considero que, en ciertas ocasiones, se puede utilizar.

1.7.4 La función de biblioteca `printf`

Como ya sabemos la función `printf` permite mostrar datos en la consola. La salida puede estar compuesta por un texto fijo (texto literal) o por una combinación de texto literal y contenido variable como veremos en el siguiente programa.

```
#include <stdio.h>

int main()
{
    char nombre[] = "Pablo";
    int edad = 39;
    double altura = 1.70;
```

```
        printf("Mi nombre es %s, tengo %d años y mido %lf metros.\n",
               ,nombre
               ,edad
               ,altura);

    return 0;
}
```

La salida de este programa es:

Mi nombre es Pablo, tengo 39 años y mido 1.70 metros.

La función `printf` intercaló los valores de las variables `nombre`, `edad` y `altura` dentro del texto literal, en las posiciones indicadas con los `%` (marcadores de posición o *placeholders*).

La cadena que contiene el texto literal con los marcadores de posición se llama “máscara” y es el primer argumento que recibe `printf`. A continuación, le pasamos tantos argumentos como *placeholders* tenga la máscara.

```
printf("Mi nombre es %s, tengo %d años y mido %lf metros.\n",
       ,nombre
       ,edad
       ,altura);
```

Cada marcador debe ir acompañado de un carácter que representa un tipo de datos. Por ejemplo, `%s` indica que allí se mostrará un valor alfanumérico, `%d` representa a un valor entero y `%lf` indica que se mostrará un valor flotante.

Por último, pasamos la lista de variables cuyos valores se intercalarán en el texto que queremos que `printf` escriba en la consola.

1.7.5 La función de biblioteca `scanf`

Esta función permite leer datos a través del teclado y los asigna en las variables que le pasemos como argumento.

Analicemos el código del siguiente programa donde se le pide al usuario que ingrese datos de diferentes tipos.

```
#include <stdio.h>

int main()
{
    char nombre[20];
    int edad;
    double altura;

    printf("Ingrese su nombre: ");
    scanf("%s", nombre);

    printf("Ingrese su edad: ");
    scanf("%d", &edad);

    printf("Ingrese su altura: ");
    scanf("%lf", &altura);
```



```

    printf("Ud. es %s, tiene %d años y una altura de %lf\n"
           , nombre
           , edad
           , altura);

    return 0;
}

```

`scanf` recibe como primer argumento una máscara que indica el tipo de los datos que la función debe leer. Los marcadores coinciden con los que utiliza `printf`, por tanto con `%s` le indicamos que lea una cadena de caracteres y con `%d` y `%lf` le indicamos que lea un entero y un flotante respectivamente.

Para que una función pueda modificar el valor de un argumento tenemos que pasarle su referencia o su dirección de memoria. Esto lo obtenemos anteponiendo el operador `&` (léase “operador ampersand”) al identificador de la variable cuyo valor queremos que la función pueda modificar. Veamos las siguientes líneas:

```

printf("Ingrese su edad: ");
scanf("%d", &edad);

printf("Ingrese su altura: ");
scanf("%lf", &altura);

```

Utilizamos `scanf` para leer un valor entero y asignarlo en la variable `edad`, luego usamos `scanf` para leer un valor flotante y asignarlo en la variable `altura`.

El caso de las cadenas de caracteres es diferente. Como comentamos más arriba, las cadenas se manejan como *arrays* (o conjuntos) de caracteres. Si bien este tema lo estudiaremos más adelante, es importante saber que el identificador de un *array* es en sí mismo su dirección de memoria, razón por la cual no fue necesario anteponer el operador `&` a la variable `nombre` para que `scanf` pueda modificar su valor.

```

printf("Ingrese su nombre: ");
scanf("%s", nombre);

```

Nota: en todos los casos `scanf` lee desde el teclado valores alfanuméricos. Luego, dependiendo de la máscara, convierte estos valores en los tipos de datos que corresponda y los asigna en sus respectivas variables.

1.7.6 El operador de dirección &

El operador `&` se llama “operador de dirección” y aplicado a una variable nos permite obtener su referencia o dirección de memoria.

Tendremos que utilizar este operador cada vez que necesitemos que una función pueda modificar el valor de alguna variable que le pasemos como argumento.

1.7.7 Las constantes

Los algoritmos resuelven situaciones problemáticas que surgen de la realidad, donde existen valores que nunca cambiarán. Dos ejemplos típicos son los números `PI` y `E` cuyas aproximaciones son: 3.141592654 y 2.718281828 respectivamente.

1.7.7.1 La directiva de preprocesador `#define`

Esta directiva permite definir valores constantes de la siguiente manera:

```

#define NUMERO_PI 3.1415169254
#define NUMERO_E 2.718281828

```

El preprocesador de C reemplazará cada aparición de `NUMERO_PI` y `NUMERO_E` por sus respectivos y correspondientes valores.

1.7.7.2 El modificador `const`

Si a la declaración de una variable le aplicamos el modificador `const` su valor ya no podrá ser modificado. Por ejemplo:

```
const int temperaturaMaxima = 45;
```

Luego, cualquier intento de asignar otro valor a la variable `temperaturaMaxima` en el código del programa generará un error de compilación.

1.7.8 Nomenclatura para las constantes

Adoptaremos la siguiente convención de nomenclatura para los nombres de las constantes definidas con la directiva `#define`:

Las constantes deben escribirse completamente en mayúscula. Luego, si se trata de un nombre compuesto por dos o más palabras, cada una debe separarse de la anterior mediante el carácter guión bajo o *underscore*.

En cambio, para las constantes declaradas con el modificador `const` respetaremos la convención de nomenclatura para variables estudiada más arriba.

1.8 Operadores aritméticos

Comenzamos el capítulo explicando que los recursos que tenemos para diseñar y desarrollar algoritmos son la memoria y la capacidad de ejecutar operaciones aritméticas y lógicas.

Todos los lenguajes de programación proveen un conjunto de operadores con los que podemos realizar operaciones aritméticas. En C estos operadores son los siguientes:

Operador	Descripción
+	suma
-	resta
*	multiplicación
/	división (cociente)
%	módulo, resto o valor residual

Veamos un ejemplo.

Problema 1.1

Leer dos valores enteros e informar su suma.

Análisis

En este problema, los datos de entrada son los dos valores numéricos enteros que ingresará el usuario. La salida del algoritmo será un mensaje en la consola informando la suma de estos valores. El proceso implica sumar los dos valores y mostrar el resultado en la pantalla.

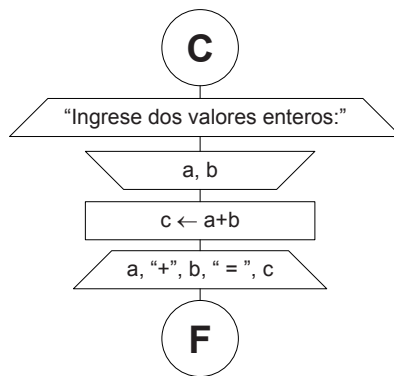


Fig. 1.12 Suma dos valores numéricos.

Comenzamos el algoritmo emitiendo un mensaje para informarle al usuario que debe ingresar dos valores numéricos enteros. A continuación, leemos los valores que el usuario va a ingresar, quedarán asignados en las variables `a` y `b`. Luego asignamos en la variable `c` la suma `a+b` y mostramos el resultado intercalando los valores de las variables `a`, `b` y `c` con las cadenas literales `“+”` e `“=”`.

La codificación del algoritmo es la siguiente:

```
#include <stdio.h>

int main()
{
    int a,b,c;

    printf("Ingrese dos valores enteros: ");
    scanf("%d %d",&a, &b);

    c = a+b;

    printf("%d + %d = %d\n",a,b,c);

    return 0;
}
```

El algoritmo hubiera sido, prácticamente, el mismo si en lugar de tener que mostrar la suma de los dos valores ingresados por el usuario nos hubiera pedido que mostremos su diferencia o su producto. Pero, ¿qué sucedería si nos pidieran que mostremos su cociente? Lo analizaremos a continuación:

Problema 1.2

Leer dos valores numéricos enteros e informar su cociente.

Análisis

En este problema, los datos de entrada son los dos valores enteros que ingresará el usuario a través del teclado (los llamaremos `a` y `b`) y la salida será su cociente (un número flotante).

Ahora bien, existe la posibilidad de que el usuario ingrese como segundo valor el número 0 (cero). En este caso, no podremos mostrar el cociente ya que la división por cero es una indeterminación, así que tendremos que emitir un mensaje informando las causas por las cuales no se podrá efectuar la operación.

Para resolver este algoritmo utilizaremos una estructura condicional que nos permitirá decidir, en función del valor de b , entre mostrar un mensaje de error y realizar la división e informar el cociente. Veamos el algoritmo:

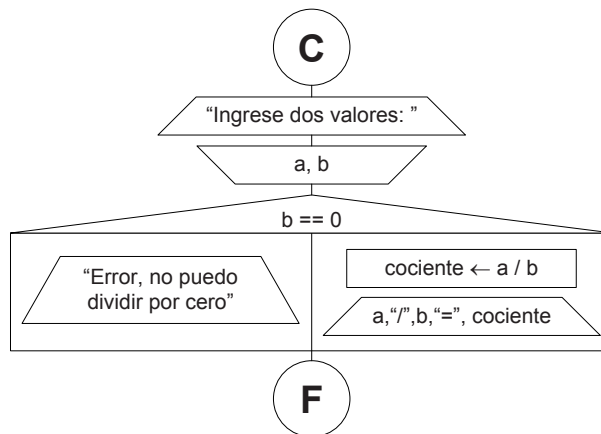


Fig. 1.13 Muestra el cociente de la división de dos números.

Leemos dos valores en las variables a y b y luego preguntamos si el valor de b es cero. Para esto, utilizamos el operador de comparación $==$ (léase “igual igual”). Si efectivamente b vale cero entonces mostramos un mensaje de error informando lo sucedido, si no asignamos a la variable `cociente` el resultado de la división a/b y lo mostramos en la consola. El código fuente es el siguiente:

```
#include <stdio.h>

int main()
{
    int a,b;
    double cociente;

    printf("Ingrese dos valores: ");
    scanf("%d %d", &a, &b);

    // verifico si el denominador es cero
    if( b == 0 )
    {
        printf("Error, no puedo dividir por cero\n");
    }
    else
    {
        cociente = (double)a/b;
        printf("%d / %d = %lf\n",a,b,cociente);
    }

    return 0;
}
```

Como vemos, la estructura de decisión se codifica con `if` (“si” en inglés). Si se verifica la condición expresada dentro de los paréntesis del `if` el programa ingresará por el primer bloque de código para mostrar el mensaje de error. Si la condición no se verifica (`else`) el programa ingresará al segundo bloque de código para efectuar la división y mostrar el resultado.

1.8.1 Conversión de tipos de datos (type casting)

C permite convertir datos de un tipo a otro. Esta operación se llama *type casting* o simplemente *casting*.

Si bien la conversión de datos de un tipo a otro es automática, en ocasiones necesitaremos realizarla explícitamente.

Observemos la línea de código donde calculamos la división y asignamos el resultado a la variable `cociente`:

```
cociente = (double)a/b;
```

El operador `/` (operador de división) convierte el resultado al mayor tipo de datos de sus operandos. Esto significa que si estamos dividiendo dos `int` entonces el resultado también será de tipo `int`, por lo tanto, el cociente que obtendremos será el de la división entera.

Para solucionar este problema y no perder los decimales, tenemos que convertir el tipo de alguno de los operandos a `double` (el tipo de datos de la variable `cociente`). Así, el mayor tipo de datos entre `int` y `double` es `double` por lo que el resultado de la división también lo será.

Le recomiendo al lector eliminar el *casteo* y luego recompilar y probar el programa para observar que resultados arroja.

1.8.2 El operador `%` (“módulo” o “resto”)

El operador `%` (léase “operador módulo” u “operador resto”) retorna el resto (o valor residual) que se obtiene luego de efectuar la división entera de sus operandos.

Por ejemplo:

```
int a = 5;
int b = 3
int r = a % b;
```

En este ejemplo estamos asignando a la variable `r` el valor 2 ya que este es el resto (o valor residual) que se origina al dividir 5 por 3.

Problema 1.3

Dado un valor numérico entero, informar si es par o impar.

Análisis

En este problema tenemos un único dato de entrada: un valor numérico entero que deberá ingresar el usuario. La salida del algoritmo será informar si el usuario ingresó un valor par o impar.

Sabemos que un número par es aquel que es divisible por 2 o, también, que un número es par si el valor residual que se obtiene al dividirlo por 2 es cero.

Según lo anterior, podremos informar que el número ingresado por el usuario es par si al dividirlo por 2 obtenemos un resto igual a cero. De lo contrario, informaremos que el número es impar.

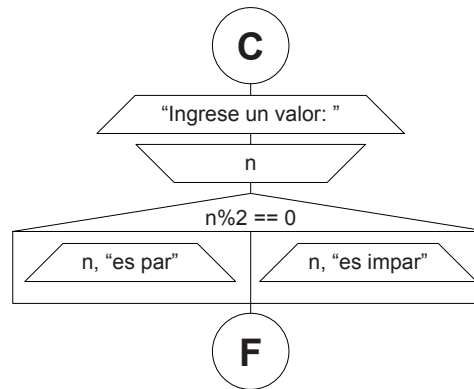


Fig. 1.14 Informa si un número es par o impar.

El código fuente es el siguiente:

```
#include <stdio.h>

int main()
{
    int n;

    printf("Ingrese un valor: ");
    scanf("%d",&n);

    if( n%2 == 0 )
    {
        printf("%d es par\n",n);
    }
    else
    {
        printf("%d es impar\n",n);
    }

    return 0;
}
```

Veamos otro ejemplo en el cual tendremos que utilizar los operadores aritméticos de módulo y división.

Problema 1.4

Se ingresa un valor numérico de 8 dígitos que representa una fecha con el siguiente formato: *aaaammdd*. Esto es: los 4 primeros dígitos representan el año, los siguientes 2 dígitos representan el mes y los 2 dígitos restantes representan el día. Se pide informar por separado el día, el mes y el año de la fecha ingresada.

Análisis

El dato que ingresará al algoritmo es un valor numérico de 8 dígitos como el siguiente: 20081015. Si este fuera el caso, entonces la salida deberá ser:

```
dia: 15
mes: 10
anio: 2008
```

Es decir, el problema consiste en desmenuzar el número ingresado por el usuario para separar los primeros 4 dígitos, después los siguientes 2 dígitos y luego, los 2 últimos.

Supongamos que efectivamente el valor ingresado es 20081015 entonces si lo dividimos por 10 obtendremos el siguiente resultado:

$$20081015 / 10 = 2008101,5$$

Pero si en lugar de dividirlo por 10 lo dividimos por 100 el resultado será:

$$20081015 / 100 = 200810,15$$

Siguiendo el mismo razonamiento, si al número lo dividimos por 10000 entonces el resultado que obtendremos será:

$$20081015 / 10000 = 2008,1015$$

Si de este valor tomamos solo la parte entera tendremos 2008 que coincide con el año representado en la fecha que ingresó el usuario.

Por otro lado, el resto obtenido en la división anterior será: 1015. Esto coincide con el mes y el día, por lo tanto, aplicando un razonamiento similar podremos separar y mostrar estos valores.

Para representar la “división entera” en los diagramas, utilizaremos el operador `div`. Este operador no existe en C, pero nos permitirá diferenciar, visualmente, entre una división real o flotante y una división entera.

Veamos el algoritmo:

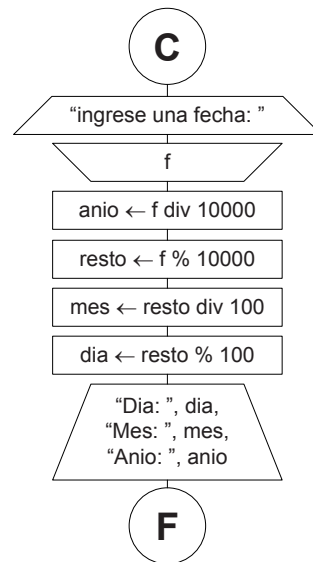


Fig. 1.15 Separa los dígitos de un número entero.

Para codificar este algoritmo debemos tener en cuenta que el valor que ingresará el usuario representa una fecha con formato `aaaammdd`. Es decir que, en el peor de los casos, este valor será 99991231 y no lo podemos almacenar en 2 bytes ya que excede por mucho su capacidad (32767 o 65535). Por este motivo, utilizaremos 4 bytes que nos permitirán almacenar números de hasta $2^{32}-1$ que superan los 8 dígitos, es decir: trabajaremos con variables de tipo `long`.

Recordemos que, por cuestiones didácticas, consideramos que el tipo `int` representa 2 bytes y el tipo `long` representa 4 bytes.
El código fuente es el siguiente:

```
#include <stdio.h>

int main()
{
    long f;
    int dia, mes, anio;
    int resto;

    printf("Ingrese una fecha: ");
    scanf("%ld",&f);

    // recordemos que la division entre dos enteros tambien lo sera
    anio = f/10000;
    resto = f%10000;

    mes = resto/100;
    dia = resto%100;

    printf("Dia: %d\n",dia);
    printf("Mes: %d\n",mes);
    printf("Anio: %d\n",anio);

    return 0;
}
```

Recordemos que el operador de división `/` retorna un resultado del mismo tipo de datos que el mayor tipo de sus operandos. Como, en este caso, `f` es de tipo `long` entonces el cociente también lo será. Es decir que la división será entera.

1.8.3 Operadores relacionales

Estos operadores permiten evaluar la relación que existe entre dos valores numéricos. Ya hemos mencionado y utilizado al operador de comparación `==` (igual igual) para comparar si un número es igual a otro. Ahora veremos la lista completa que incluye a los operadores “mayor”, “menor”, “mayor o igual”, “menor o igual” y “distinto”.

Operador	Descripción
>	mayor que...
<	menor que...
>=	mayor o igual que...
<=	menor o igual que...
==	igual a...
!=	distinto de...

Como veremos más adelante, el operador `!` (signo de admiración) es el operador lógico de negación (operador “*not*”), por lo tanto, el operador `!=` puede leerse como “distinto”, “no igual” o “*not equals*”.

1.9 Expresiones lógicas

Llamamos expresión lógica a una proposición que es susceptible de ser verdadera o falsa. Es decir, una proposición que tiene valor de verdad.

Por ejemplo, las siguientes proposiciones son expresiones lógicas cuyo valor de verdad es verdadero:

- La Tierra gira alrededor del Sol.
- EE. UU. tiene dos costas.
- 2 “es menor que” 5 (o simplemente $2 < 5$).
- $5 + 1 = 6$

Y las siguientes proposiciones son expresiones lógicas cuyo valor de verdad es falso:

- La Tierra es el centro del universo.
- EE. UU. queda en Europa.
- 2 “es mayor que” 5 (o simplemente $2 > 5$).
- $5 + 1 = 8$

En cambio, no son expresiones lógicas las siguientes proposiciones:

- Hoy hace frío.
- La pared está bastante sucia.
- Los números impares tienen mejor “Chi”.

Las expresiones lógicas pueden combinarse entre sí formando nuevas expresiones lógicas con su correspondiente valor de verdad. Para esto, se utilizan los operadores lógicos.

1.9.1 Operadores lógicos

Las expresiones lógicas pueden conectarse a través de los operadores lógicos y así se obtienen expresiones lógicas compuestas cuyo valor de verdad dependerá de los valores de verdad de las expresiones lógicas simples que las componen.

Los operadores lógicos son los siguientes:

Operador	Descripción
$\&$	“and” o producto lógico
$\mid$	“or” o suma lógica
$!$	“not” o negación

Con los operadores lógicos podemos conectar dos o más expresiones lógicas y así obtener una nueva expresión lógica con su correspondiente valor de verdad.

Sean las expresiones lógicas p y q cada una con su correspondiente valor de verdad entonces el valor de verdad de la expresión lógica $h = p \ \&\ q$ será verdadero o falso según la siguiente tabla:

p	q	$h = p \ \&\ q$
verdadero	verdadero	verdadero
verdadero	falso	falso
falso	verdadero	falso
falso	falso	falso



El Feng Shui explica como nos afectan los flujos energéticos de nuestro entorno, dicha energía es conocida como Chi, circula por los diferentes espacios afectando en su recorrido todo lo que toca.

Es decir, si p es verdadero y q es verdadero entonces la expresión lógica $p \ \&\& \ q$ también lo es. En cambio, si alguna de las dos expresiones lógicas (o las dos) es falsa entonces su producto lógico también lo será.

Veamos ahora la tabla del operador $||$ (operador “or”):

p	q	$h = p \ \ q$
verdadero	verdadero	verdadero
verdadero	falso	verdadero
falso	verdadero	verdadero
falso	falso	falso

Como vemos, la suma lógica siempre resulta verdadera a no ser que el valor de verdad de los dos operandos sea falso.

El operador $!$ (operador “not”) niega el valor de verdad del operando. Es decir, si el operando es verdadero entonces su negación será falsa. En cambio, si el operando es falso su negación será verdadera.

p	$h = !p$
verdadero	falso
falso	verdadero

1.10 Operadores de bits

En este apartado, estudiaremos operadores que permiten manipular los bits de los *bytes* vinculados a las variables enteras que declaramos en los programas.

La complejidad del tema excede a la que venimos manejando en este capítulo introductorio razón por la cual le recomiendo al lector pasar por alto esta lectura ya que, oportunamente, cuando lo considere adecuado le recomendaré retomarla.

1.10.1 Representación binaria de los tipos enteros

Más arriba estudiamos que un valor numérico entero se puede representar en un conjunto de *bytes*. Cuanto más grande sea la cantidad de *bytes* de este conjunto, mayor será la diversidad de valores enteros que permitirá abarcar.

En el lenguaje de programación C, estos conjuntos de *bytes* están representados por los tipos de datos: `char`, `short`, `int` y `long`.

Para facilitar el análisis, nos manejaremos con valores pequeños desarrollando ejemplos basados en el tipo `char` que, como ya sabemos, representa un conjunto de un único *byte* con bit de signo.

Sea la variable `c` definida de la siguiente manera:

```
char c = 38;
```

Internamente, su representación será:

00100110

Luego, si multiplicamos su valor por -1:

```
c = -1*c;
```

Su representación pasará a ser:

11011010

y la obtenemos al invertir todos los bits del número positivo y luego, sumando 1 al resultado.

1.10.2 Operadores de desplazamiento de bits (>> y <<)

Los operadores de desplazamiento (*shift* en inglés) permiten mover los bits de una variable entera hacia la izquierda (*shift left*) o hacia la derecha (*shift right*).

Continuando con la variable `c` declarada más arriba, si hacemos:

```
// desplazamos los bits de c una posicion hacia la derecha
c = c>>1;
```

`c` pasará a valer 19. Veamos:

c	00100110	38
c = c>>1	00010011	19

Y si en lugar de correr sus bits hacia la derecha los corremos hacia la izquierda:

```
c = c<<1;
```

la variable pasará a valer 76.

c	00100110	38
c = c<<1	01001100	76

¿Qué sucederá si corremos los bits de `c` 2 lugares hacia la izquierda?

c	00100110	38
c = c<<2	10011000	152

El resultado que deberíamos obtener es 152. Sin embargo, como la variable es de tipo `char` solo admite valores positivos no mayores a 127. De esta manera, la asignación desbordará la capacidad de la variable y le asignará un valor absurdo e irreal.

1.10.3 Representación hexadecimal

La representación hexadecimal facilita, enormemente, la tarea de trabajar con bits (dígitos binarios) ya que un solo dígito hexadecimal permite representar 4 dígitos binarios.

La siguiente tabla muestra la combinación binaria que representa cada uno de los 16 dígitos hexadecimales.

Hexadecimal	Binario	Hexadecimal	Binario
0	0000	8	1000
1	0001	9	1001
2	0010	A	1010
3	0011	B	1011
4	0100	C	1100
5	0101	D	1101
6	0110	E	1110
7	0111	F	1111

Fig. 1.16 Codificación binaria de los dígitos hexadecimales.

Así, para representar el número binario 01111011 (123 decimal) solo serán necesarios los dígitos hexadecimales: 7B.

Binario:	0111	1011
Hexadecimal:	7	B

En C podemos utilizar el sistema hexadecimal para representar valores enteros. Para esto, tenemos que anteponer el prefijo 0x (léase “cero equis”) al número hexadecimal. Las siguientes líneas de código muestran diferentes formas de expresar el mismo valor numérico entero.

```
char a = 123;      // o 01111011 binario
char b = 0x7B;     // o 01111011 binario
```

Ahora las variables a y b tienen el mismo valor numérico: 123.

Representación hexadecimal de números enteros negativos

La representación hexadecimal de números enteros está separada de la representación binaria interna que el número adquiere cuando está en memoria. Por esto, el número 0x88 (10001000) representa al valor +136 aunque su bit más significativo sea 1. Para representar el valor entero -136, utilizaremos -0x88.

1.10.4 Representación octal

El sistema octal es alternativo al sistema hexadecimal. En este caso, cada dígito octal (0 a 7) representa un conjunto de 3 dígitos binarios.

Octal	Binario	Octal	Binario
0	000	4	100
1	001	5	101
2	010	6	110
3	011	7	111

Fig. 1.17 Codificación binaria de los dígitos octales.

C permite expresar valores octales anteponiendo al número el prefijo 0 (léase “cero”). Luego, necesitaremos 3 dígitos octales para representar el valor numérico entero 123 (01111011).

Binario:	001	111	011
Octal:	1	7	3

```
char x = 123; // 123 en decimal
char y = 0x7B; // 123 en hexadecimal
char z = 0173; // 123 en octal
```

1.10.5 Operadores lógicos de bits

Sean las siguientes variables declaradas e inicializadas como vemos a continuación:

```
unsigned char a = 0x61; // 01100001
unsigned char b = 0x2D; // 00101101
```

Podemos obtener su producto y su suma lógica de la siguiente manera:

Producto lógico			Suma lógica		
a	01100001	97	a	01100001	97
b	00101101	45	b	00101101	45
c = a&b	00100001	33	c = a b	01101101	109

Fig. 1.18 Operadores lógicos de bits.

El producto lógico entre dos valores enteros se obtiene comparando uno a uno los bits de sus posiciones homónimas. Luego, si ambos bits valen 1 el bit resultante para esa posición también tendrá este valor.

En cambio, para obtener la suma lógica alcanza con que alguno de los bits de una misma posición sea 1 para que el bit resultante también tenga este valor.

1.11 Resumen

En este capítulo, explicamos los conceptos básicos de programación proporcionándoles a los lectores que nunca antes han programado un primer acercamiento a la materia.

Entre estas nociones básicas e introductorias creo conveniente destacar los conceptos de “algoritmo y problema”, las estructuras de control que describe el teorema de la programación estructurada y la idea de variables y tipos de datos.

Además, con la ayuda de los videotutoriales, el lector habrá podido compilar sus primeros programas en el lenguaje C.

En el próximo capítulo, nos concentraremos en estudiar con mayor nivel de detalle las estructuras secuencial, condicional e iterativa, incrementando así el nivel de complejidad de los algoritmos.

1.12 Contenido de la página Web de apoyo



El material marcado con asterisco (*) solo está disponible para docentes.

1.12.1 Mapa conceptual

1.12.2 Autoevaluaciones

1.12.3 Videotutorial

1.12.3.1 Instalación y uso de Eclipse para C

1.12.4 Presentaciones*

Estructuras básicas de control y lógica algorítmica

Contenido

2.1	Introducción.....	36
2.2	Estructura secuencial	36
2.3	Estructura de decisión.....	36
2.4	Estructura de repetición.....	47
2.5	Contextualización del problema	63
2.6	Resumen.....	69
2.7	Contenido de la página Web de apoyo	69

Objetivos del capítulo

- Analizar y resolver algoritmos utilizando las estructuras de control básicas: secuencial, condicional e iterativa.
- Conocer, identificar y aplicar recursos algorítmicos: contadores, acumuladores, máximos, mínimos, etcétera.
- Hacer un seguimiento “paso a paso” del algoritmo y aprender a usar la herramienta de depuración: el *debugger*.
- Analizar y resolver problemas contextualizados.

Competencias específicas

- Dominar los conceptos básicos de la programación.
- Analizar problemas y representar su solución mediante algoritmos.
- Conocer las características principales del lenguaje C.
- Codificar algoritmos en el lenguaje de programación C.
- Compilar y ejecutar programas.
- Construir programas utilizando estructuras condicionales y repetitivas para aumentar su funcionalidad.

2.1 Introducción

En el capítulo anterior, estudiamos el teorema de la programación estructurada que describe tres estructuras básicas de control con las que demuestra que es posible resolver cualquier problema computacional.

En este capítulo, estudiaremos las diferentes implementaciones que provee C para estas estructuras y, además, analizaremos problemas con mayor grado de complejidad que nos permitirán integrar todos los conceptos que incorporamos a lo largo del Capítulo 1.

2.2 Estructura secuencial

En este caso, simplemente, haremos un breve repaso para recordar que la estructura secuencial consiste en ejecutar, secuencialmente, una acción simple detrás de otra.

Recordemos, también, que se considera acción simple a las acciones de leer, escribir, asignar valor a una variable e invocar a un módulo o función.

2.3 Estructura de decisión

La estructura de decisión permite decidir entre ejecutar uno u otro conjunto de acciones en función de que se cumpla o no una determinada condición lógica.

En el capítulo anterior, explicamos el uso del `if`. Más adelante, en este mismo capítulo, veremos que existen otras estructuras selectivas como por ejemplo la decisión múltiple (`switch`) y el `if-inline`.

Comencemos por analizar un problema extremadamente simple.

Problema 2.1

Leer dos valores numéricos enteros e indicar cuál es el mayor y cuál es el menor. Considerar que ambos valores son diferentes.

Análisis

Los datos de entrada para este problema son los dos valores que ingresará el usuario. Nuestra tarea será compararlos para determinar cuál es mayor y cuál es menor.

El enunciado dice que debemos considerar que los valores serán diferentes. Por lo tanto, para nuestro análisis la posibilidad de que los valores sean iguales no será tomada en cuenta.

Llamaremos `a` al primer valor y `b` al segundo. Para compararlos utilizaremos una estructura de decisión que nos permitirá determinar si `a` es mayor que `b`. Si esto resulta verdadero, entonces `a` será el mayor y `b` será el menor. De lo contrario, si la condición anterior resulta falsa, `b` será el mayor y `a` el menor ya que, como mencionamos más arriba, `a` y `b` no serán iguales.

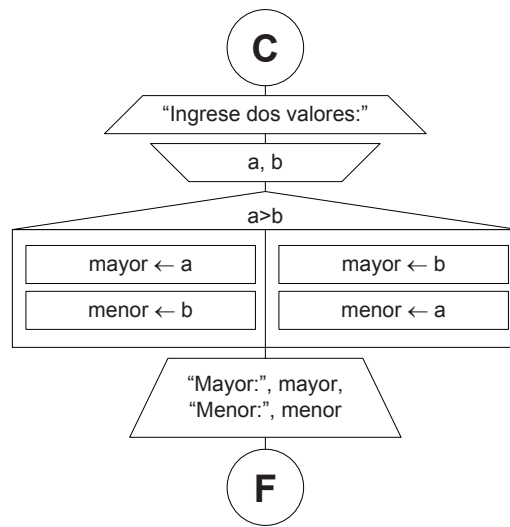


Fig. 2.1 Compara dos valores numéricos.

En el algoritmo utilizamos las variables `mayor` y `menor`. Una vez que comparamos `a` con `b` y determinamos cuál es el mayor y cuál es el menor, asignamos sus valores a estas variables y luego, mostramos sus contenidos.

El código fuente es el siguiente:

```
#include <stdio.h>

int main()
{
    int a,b;
    int mayor,menor;

    printf("Ingrese dos valores: ");
    scanf("%d %d",&a,&b);

    if( a>b )
    {
        mayor=a;
        menor=b;
    }
    else
    {
        mayor=b;
        menor=a;
    }

    printf("Mayor: %d\n",mayor);
    printf("Menor: %d\n",menor);

    return 0;
}
```


Para resolver este problema, utilizamos una estructura de decisión que nos permitió determinar cuál es el mayor valor. Luego, por descarte, el otro es el menor.

2.3.1 Estructuras de decisión anidadas

Cuando en una estructura de decisión utilizamos otra estructura de decisión, decimos que ambas son estructuras anidadas. En el siguiente problema, utilizaremos estructuras de decisión anidadas para determinar, entre tres valores numéricos, cuál es el mayor, cuál es el del medio y cuál es el menor.

Problema 2.2

Leer tres valores numéricos enteros, indicar cuál es el mayor, cuál es el del medio y cuál, el menor. Considerar que los tres valores serán diferentes.

Veamos primero el algoritmo y luego lo analizaremos.

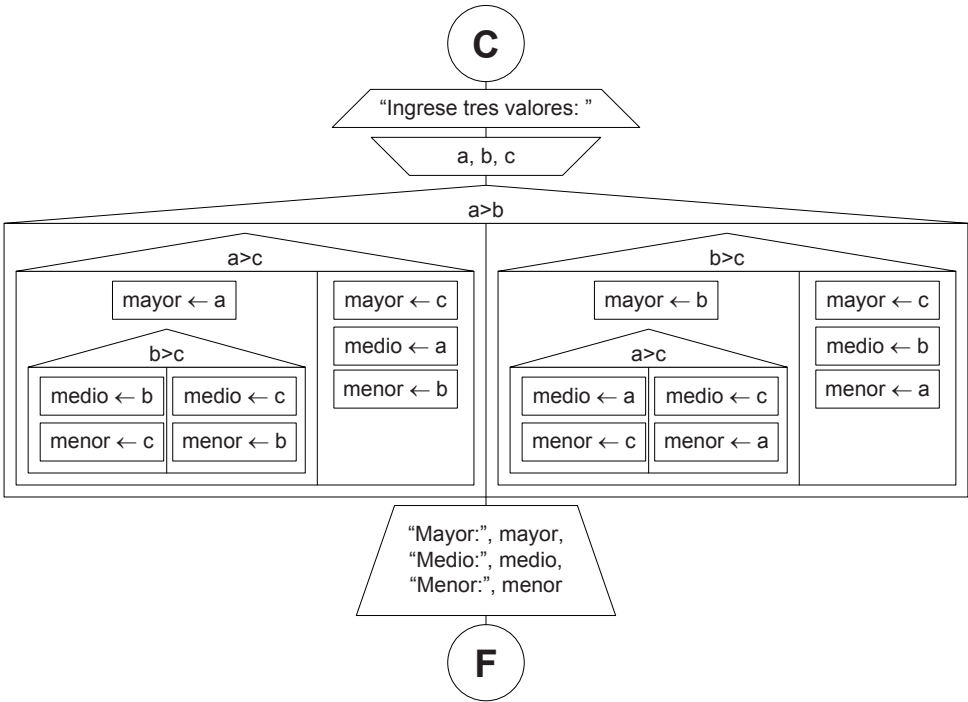


Fig. 2.2 Compara tres valores y determina cuál es mayor, medio y menor.

Análisis

Leemos los tres valores y comenzamos a comparar preguntando si $a > b$. Si esto se verifica entonces preguntamos si $a > c$. Si esto también se verifica entonces, como $a > b$ y $a > c$, no hay dudas de que a es el mayor. Luego tenemos que comparar b y c para ver cuál está en segundo y en tercer lugar.

Si resulta que $a > b$ pero no se verifica que $a > c$ (es decir que c es mayor que a) será porque c es el mayor, a el medio y b el menor.

Por otro lado, si no se verifica que $a > b$ preguntamos si $b > c$. Si esto es así, entonces el mayor será b (ya que b es mayor que a y b es mayor que c). Preguntamos si $a > c$ y podremos deducir cuál está en segundo y tercer lugar.